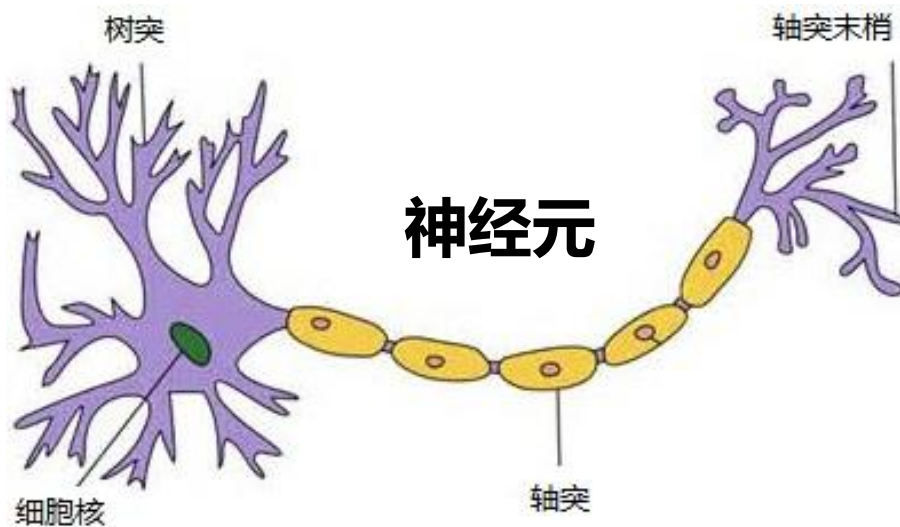


1. 神经网络基础

神经网络：是由具有适应性的**简单单元**组成的广泛并行互连的**网络**，它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应

神经网络是一种模拟人脑的神经网络以期能够实现类人工智能的机器学习技术，它是目前最为火热的研究方向--深度学习的基础



1. 神经网络基础

萌芽期（20世纪40年代）

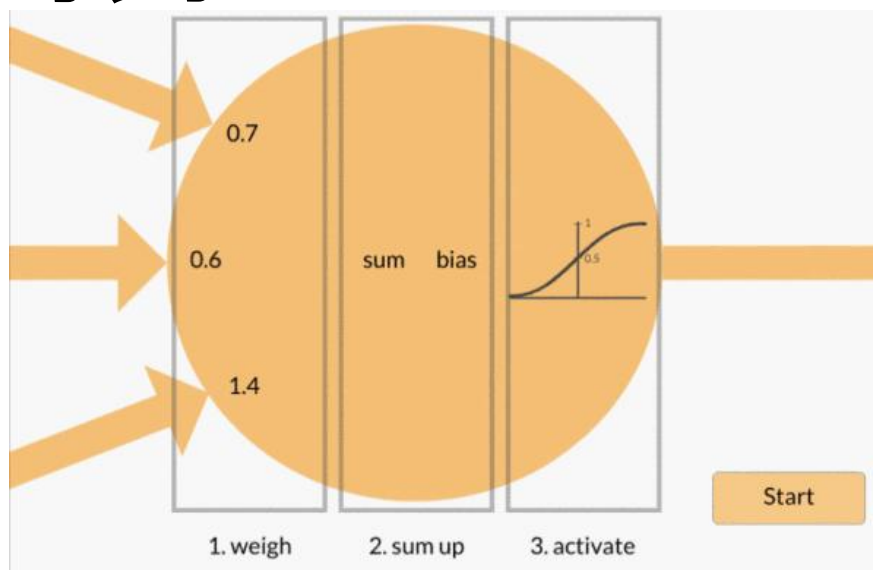
人工神经网络的研究最早可以追溯到人类开始研究自己的智能的时期，到1949年止。

- 1943年，心理学家McCulloch和数学家Pitts建立起了著名的阈值加权和模型，简称为**M-P模型**。发表于数学生物物理学会刊《Bulletin of Mathematical Biophysics》
- 1949年，心理学家D. O. Hebb提出神经元之间突触连接强度是可变的假说——**Hebb学习律**，这为后面的学习算法奠定了基础。

1. 神经网络基础

第一高潮期 (1950~1968)

- 代表人物M. Minsky, F. Rosenblatt, B. Widrow等，代表作是单级**感知器** (Perceptron)
- 可用电子线路模拟
- 乐观地认为找到了智能的关键，大批地投入研究
- “异或”运算不可表示



1. 神经网络基础

第二高潮期（1983~1990）

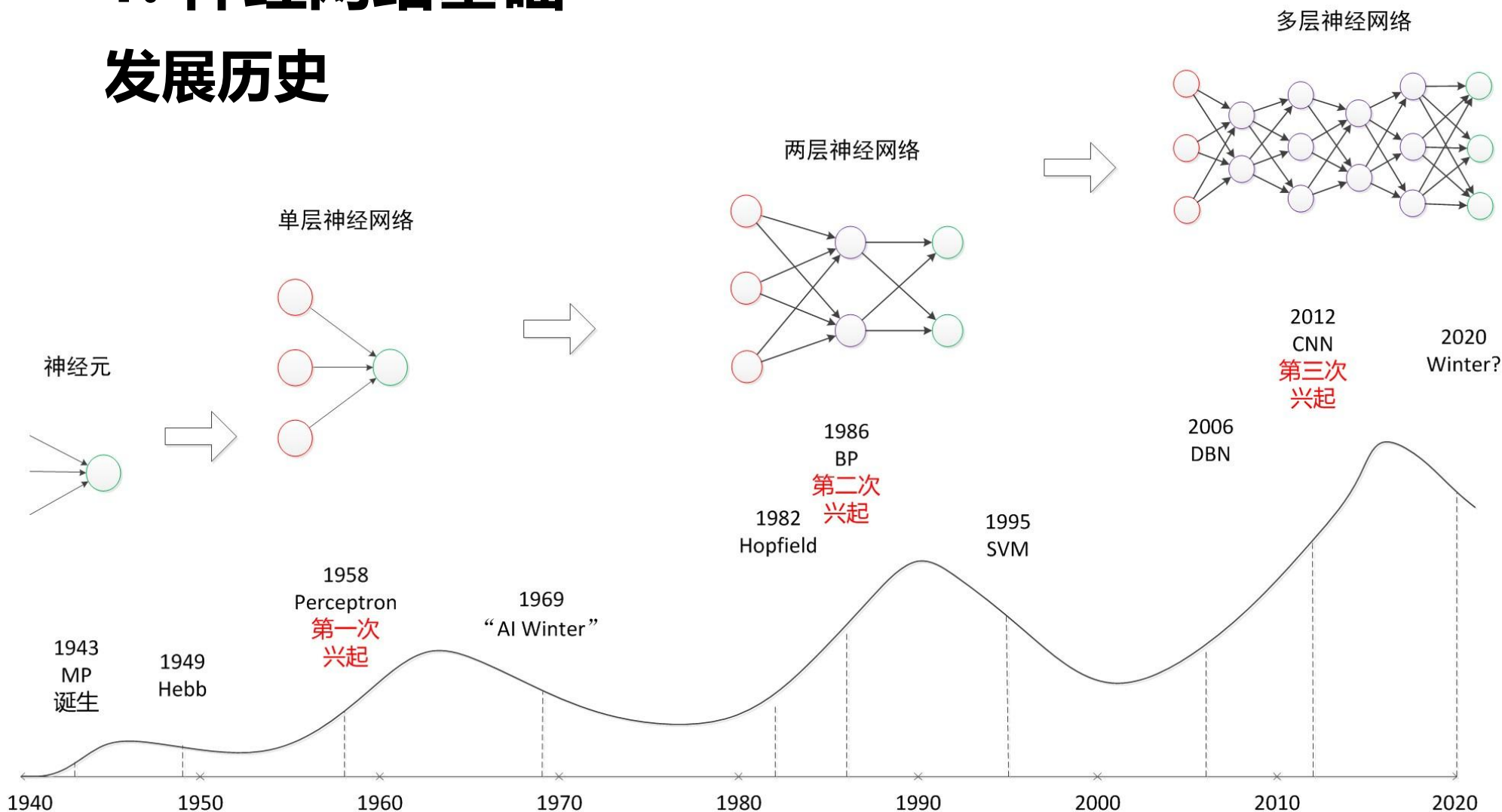
- 1982年，J. Hopfield提出**Hopfield网络**
- 1985年，UCSD的Hinton等人所在的并行分布处理（PDP）小组的研究者在Hopfield网络中引入了随机机制，提出所谓的**Boltzmann机**
- 1986年，并行分布处理小组的Rumelhart等研究者重新独立地提出多层网络的学习算法——**BP算法**，较好地解决了多层网络的学习问题
- 1989年，LeCun等人提出了卷积神经网络
- 计算复杂，梯度消失问题

1. 神经网络基础

第三高潮期（2006~至今）

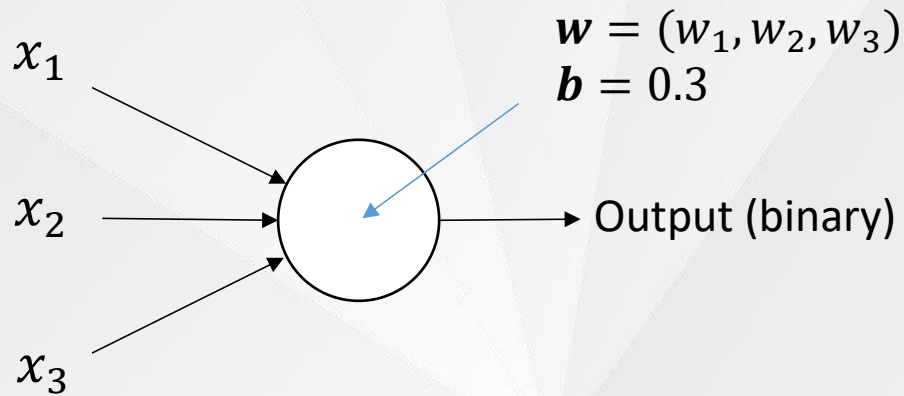
- 2006年(深度学习元年) Hinton提出了深层网络训练中梯度消失问题的解决方案：无监督预训练对权值进行初始化+有监督训练微调
- 2011年，ReLU激活函数被提出，该激活函数能够有效的抑制梯度消失问题。
- 2012年，Hinton课题组提出AlexNet网络，获得ImageNet图像识别比赛冠军
- 深度网络VGG，GoogLeNet，ResNet相继被提出

1. 神经网络基础 发展历史



感知机

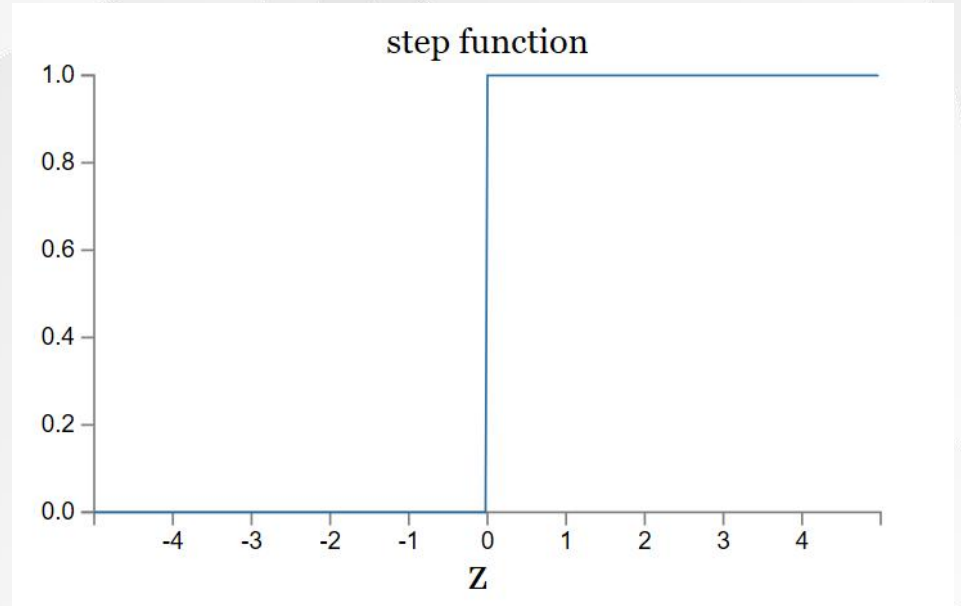
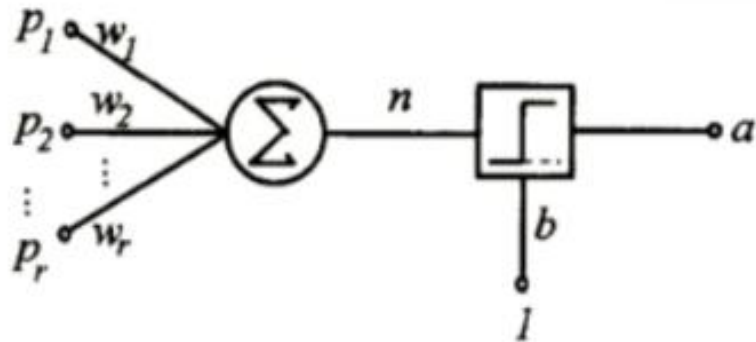
- 权重 w ,
- 偏置 b



$$w \cdot x \equiv \sum_j w_j x_j$$

$$f(x) = \text{sign}(w \cdot x + b)$$

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

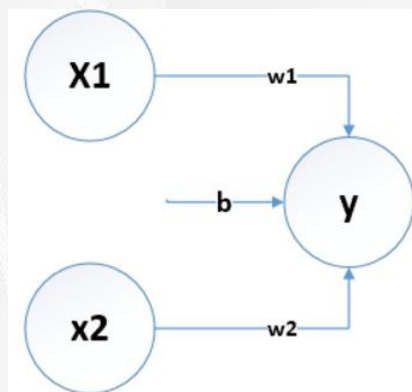


一、与门 (AND gate)

真值表

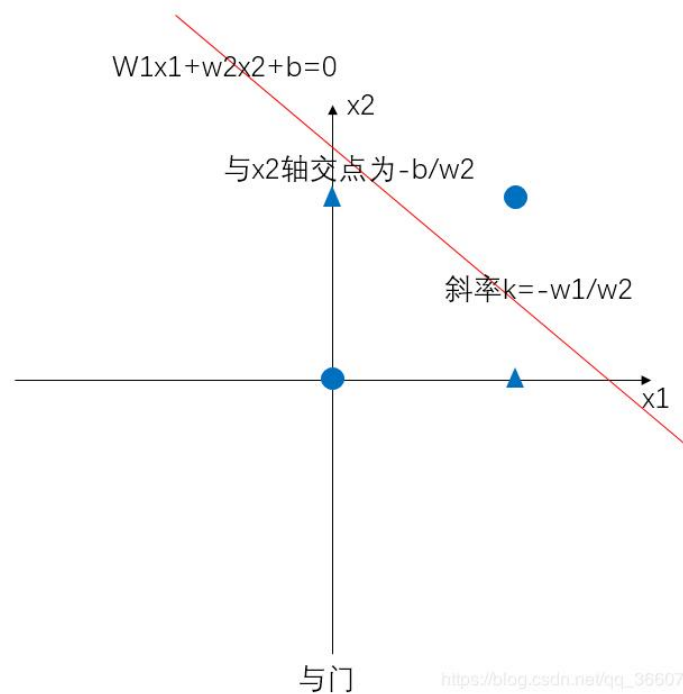
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

满足这组关系的 w_1, w_2, b 很多, 我们取0.5, 0.5, -0.7



$$y = \begin{cases} 0, & x_1 w_1 + x_2 w_2 + b \leq 0 \\ 1, & x_1 w_1 + x_2 w_2 + b > 0 \end{cases}$$

$$\begin{cases} b \leq 0 \\ w_2 + b \leq 0 \\ w_1 + b \leq 0 \\ w_1 + w_2 + b > 0 \end{cases}$$



三、与非门(NAND gate)

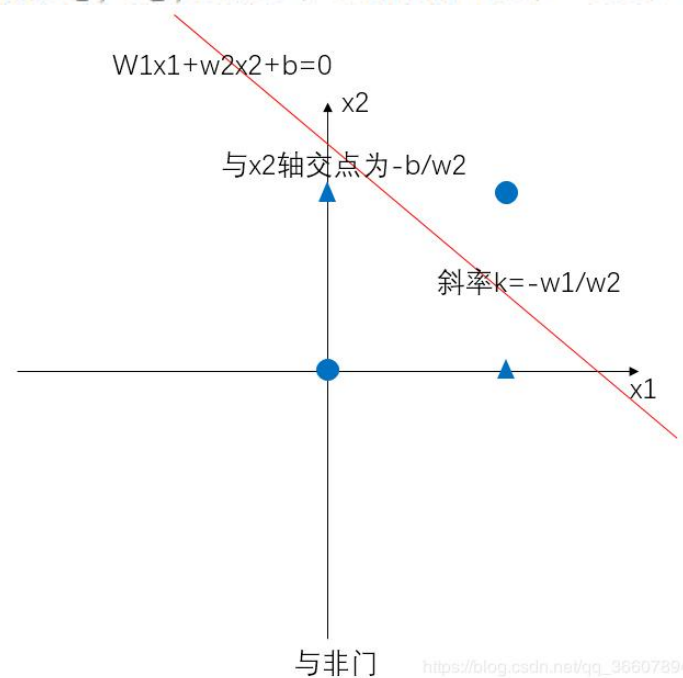
真值表

x_1	x_2	y
0	0	1
0	1	1
1	0	1
1	1	0

$$y = \begin{cases} 0, & x_1 w_1 + x_2 w_2 + b \leq 0 \\ 1, & x_1 w_1 + x_2 w_2 + b > 0 \end{cases}$$

满足这组关系的 w_1, w_2, b 很多, 我们取-0.5, -0.5, 0.7

$$\begin{cases} b > 0 \\ w_2 + b > 0 \\ w_1 + b > 0 \\ w_1 + w_2 + b \leq 0 \end{cases}$$



二、或门(OR gate)

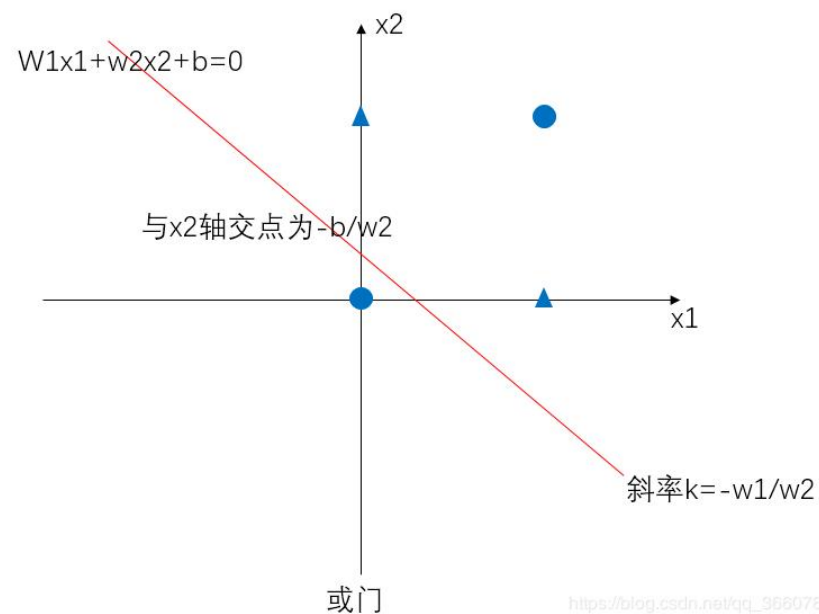
真值表

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

$$y = \begin{cases} 0, & x_1 w_1 + x_2 w_2 + b \leq 0 \\ 1, & x_1 w_1 + x_2 w_2 + b > 0 \end{cases}$$

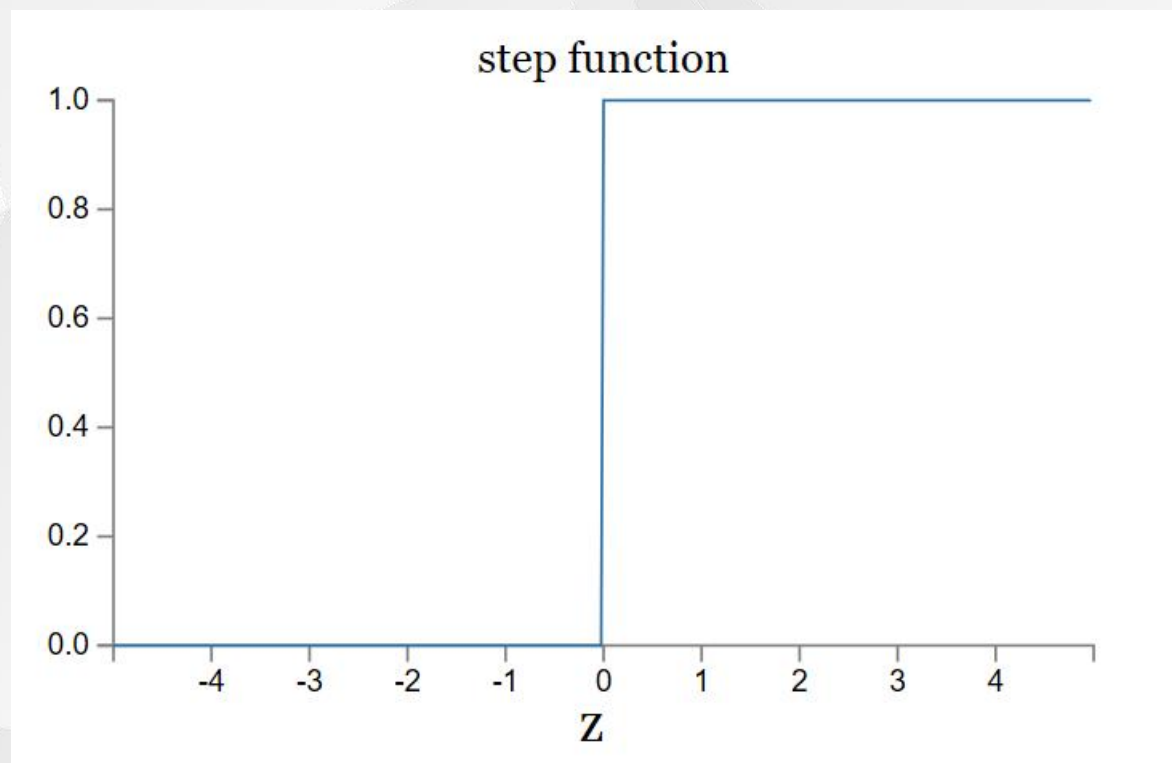
满足这组关系的 w_1, w_2, b 很多, 我们取0.5, 0.5, -0.2

$$\begin{cases} b \leq 0 \\ w_2 + b > 0 \\ w_1 + b > 0 \\ w_1 + w_2 + b > 0 \end{cases}$$



线性感知机的问题

在0附近的微小变化会引起输出的巨大不同

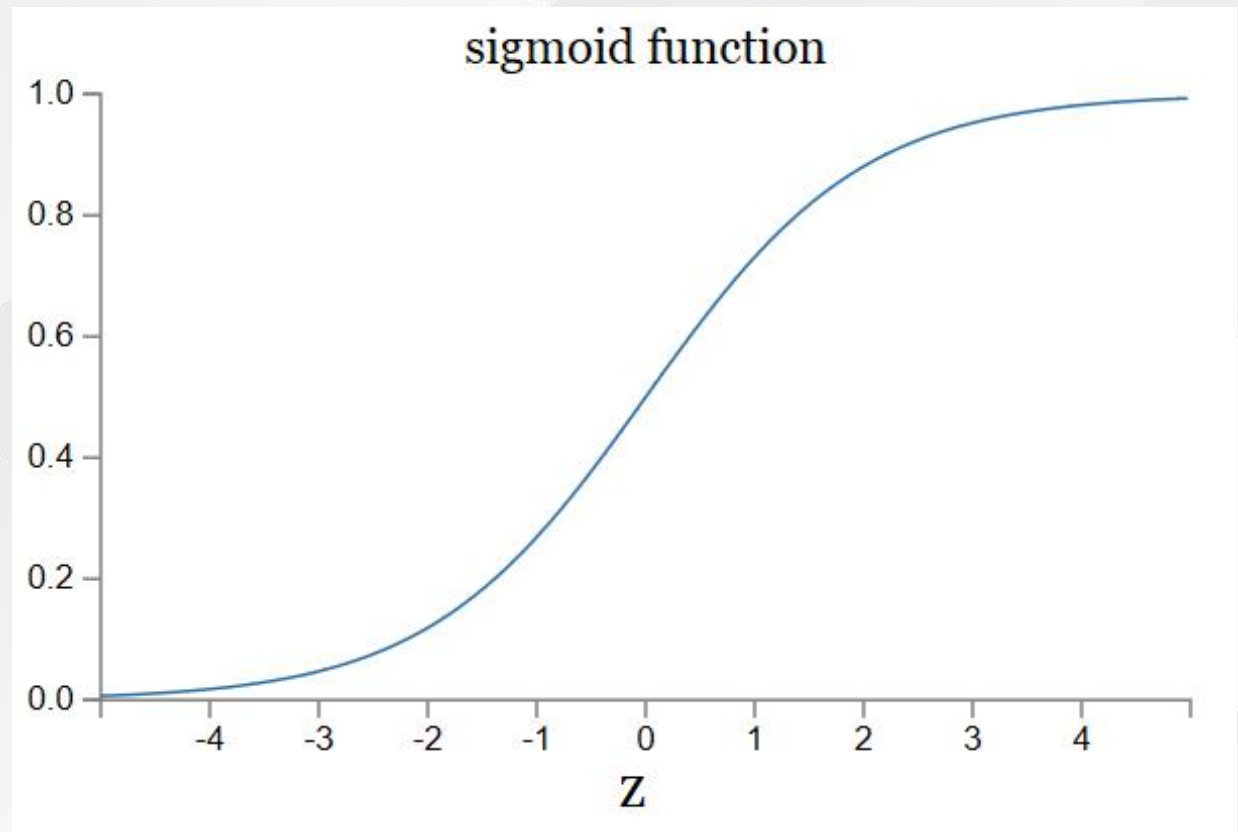


激活函数

- $y = f(wx + b)$
- 输入、参数 w, b 的微小变化引起输出的变化也很小

$$\sigma(w \cdot x + b)$$

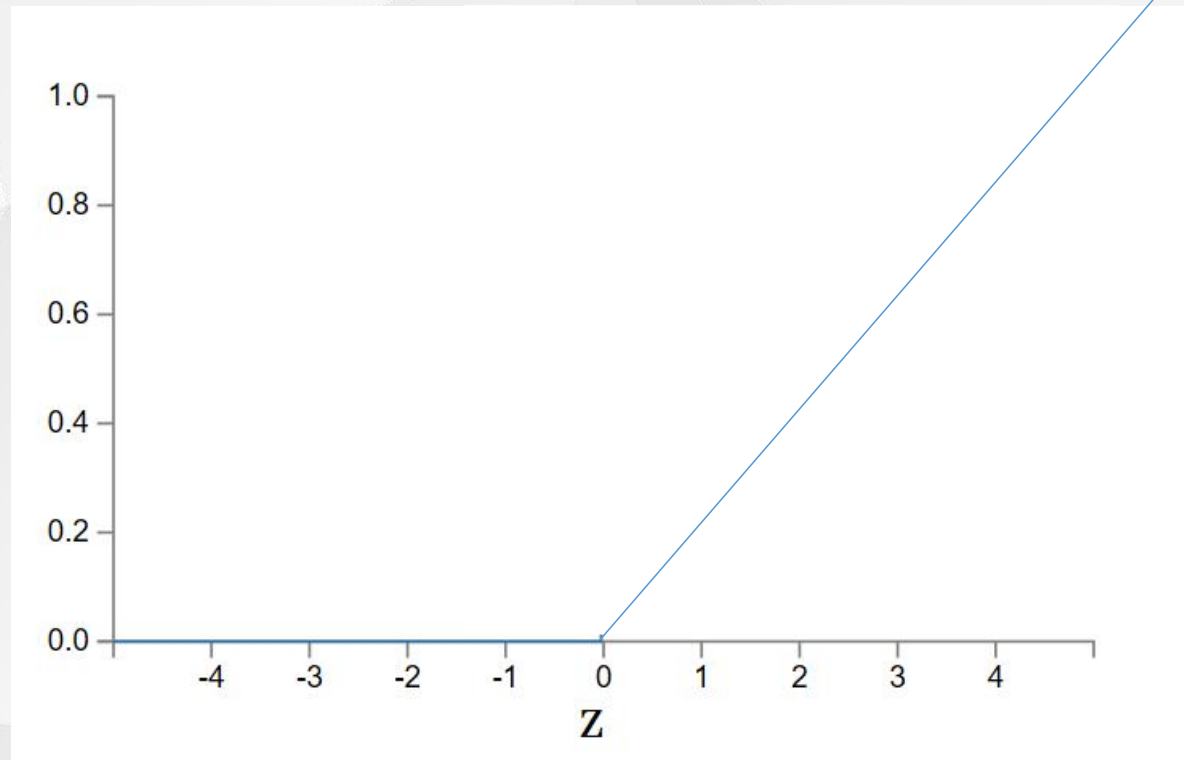
$$\sigma(z) \equiv \frac{1}{1 + e^{-z}}$$

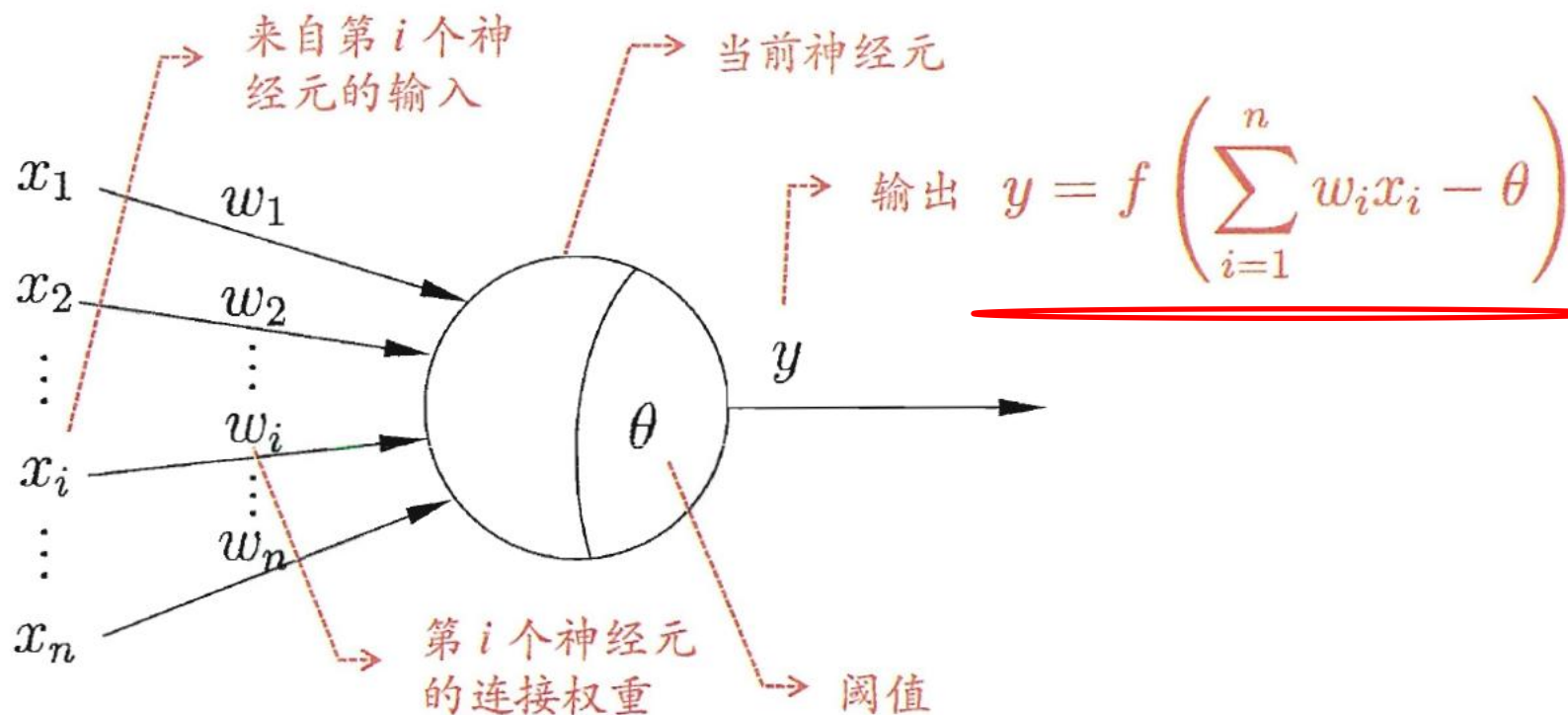


Rectified Linear Unit

- ReLU

$$f(x) = \max(0, x)$$



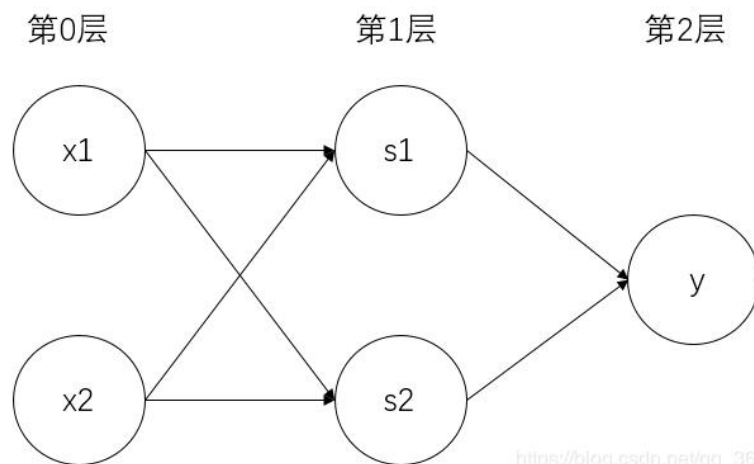
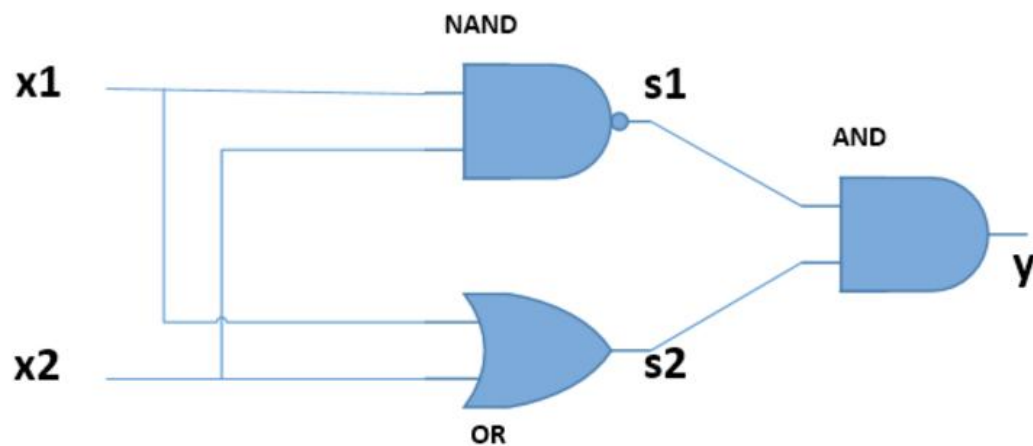


w_i 为连接权重， n 为输入信号数目， θ 为神经元单元的偏置bias (阈值)， $f()$ 为激活函数(Activation Function)， y 为神经元输出

四、异或门

真值表

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

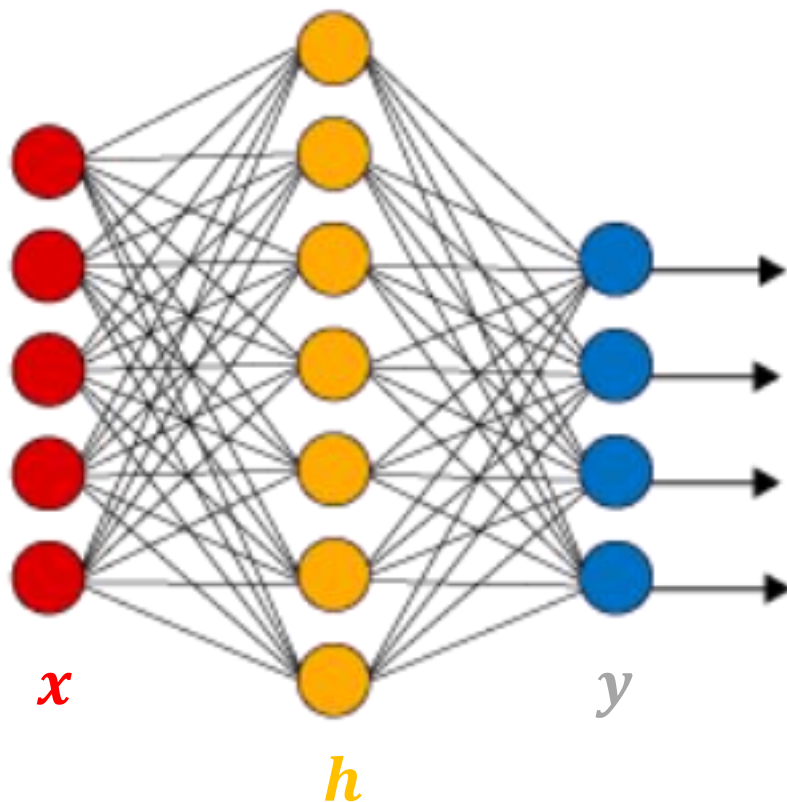


https://blog.csdn.net/qq_36607894

由NAND,OR,AND搭建XOR门电路



多层前馈神经网络(feedforward)



$$h = f(w_i x + \theta_i)$$

$$y = f(w_j h + \theta_j)$$

7 + 4 = 11 个神经元 (不计输入)

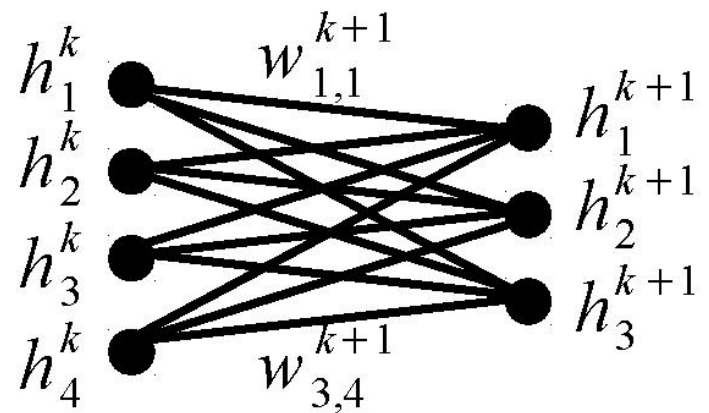
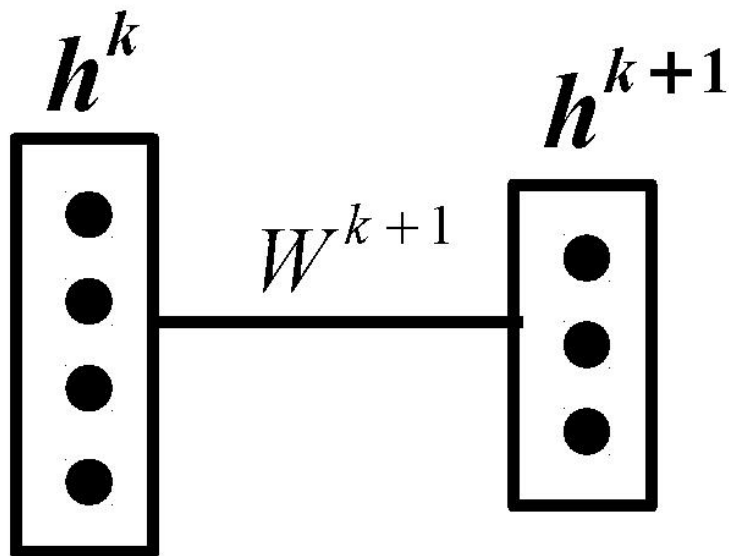
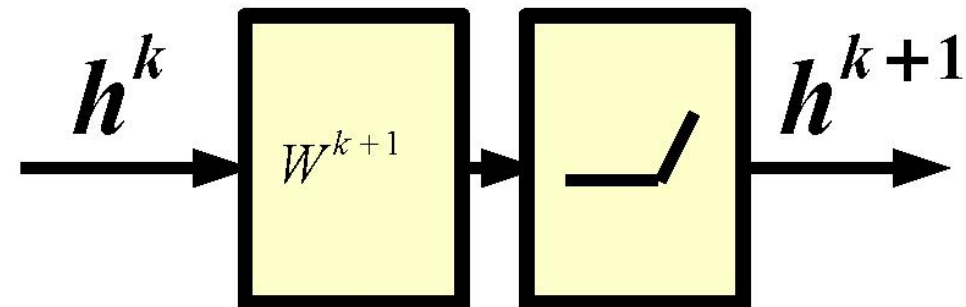
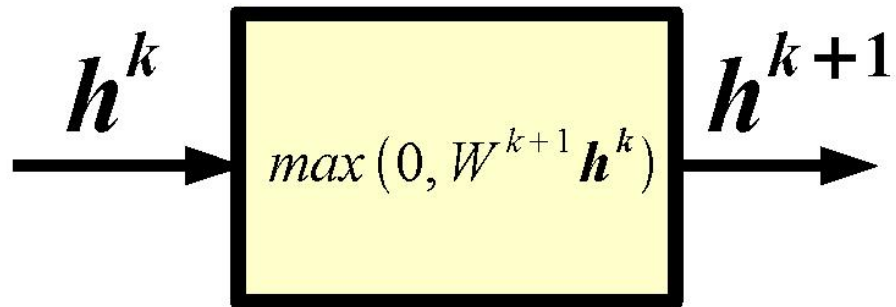
$[5 \times 7] + [7 \times 4] = 63$ 权重参数

7 + 4 = 11 偏置项

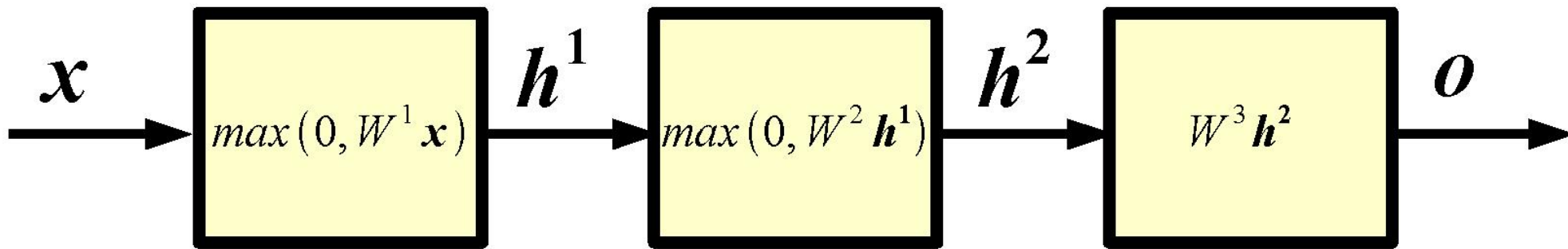
74 需学习参数

[Demo](#)

Alternative Graphical Representation



Neural Networks: example



x input

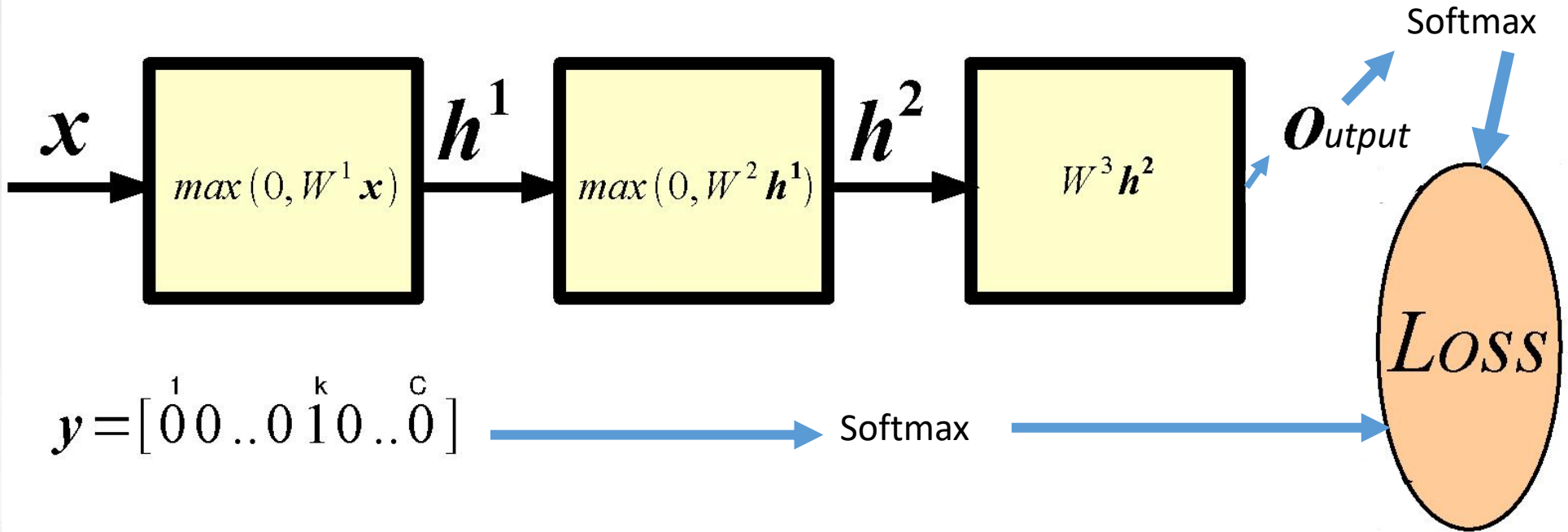
h^1 1-st layer hidden units

h^2 2-nd layer hidden units

o output

Example of a 2 hidden layer neural network (or 4 layer network, counting also input and output).

How Good is a Network?



Probability of class k given input (softmax):

$$p(c_k = 1 | \mathbf{x}) = \frac{e^{o_k}}{\sum_{j=1}^C e^{o_j}}$$

2. 经典神经网络

反向传播算法(BackPropagation)

[反向传播参考资料](#)

目标函数 $J(\theta)$

$$\theta_j^{new} = \theta_j^{old} - \eta \frac{dJ}{d\theta_j^{old}} J(\theta) \quad \text{更新每个参数}\theta$$

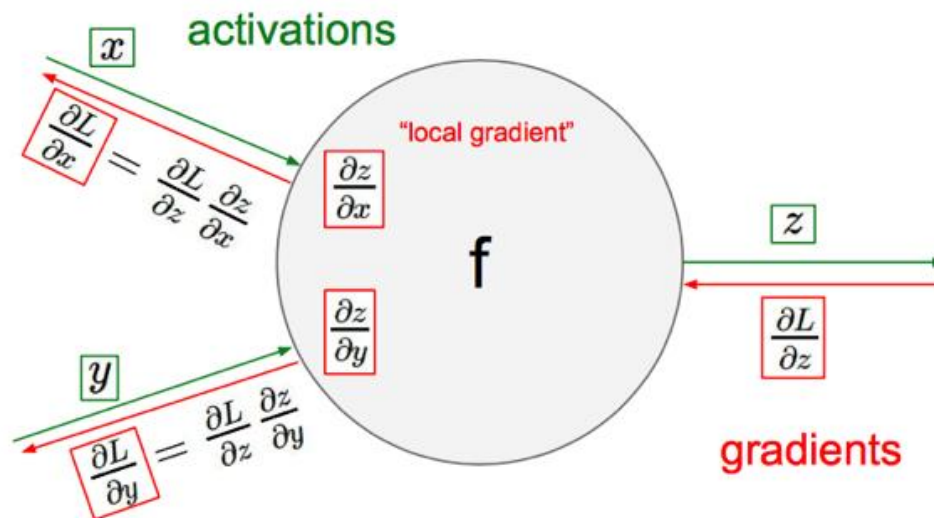
$$\theta^{new} = \theta^{old} - \eta \nabla_{\theta} J(\theta)$$

学习率

对每个结点迭代应用链式规则

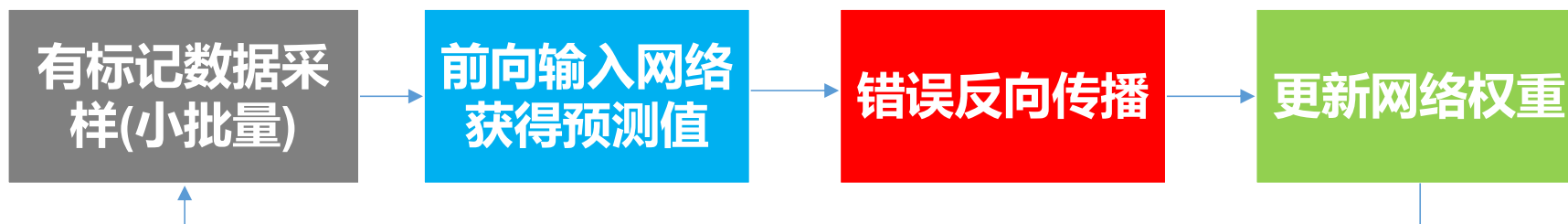
梯度下降

所有参数矩阵表示



2. 经典神经网络

神经网络训练过程

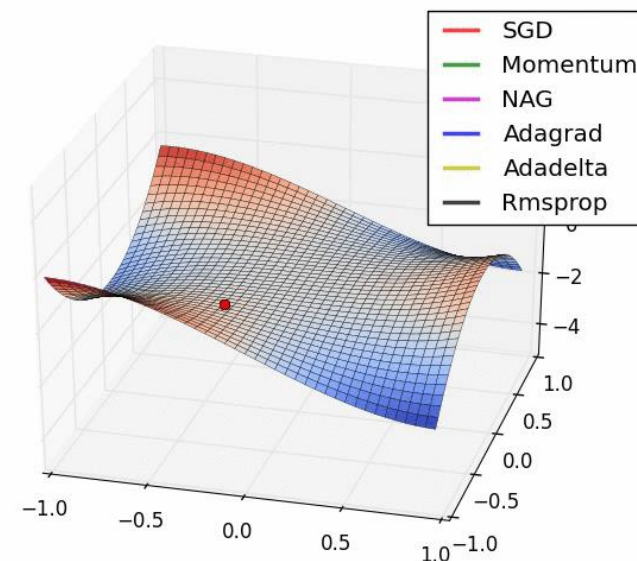


最优化 (min. 或 max.) 目标函数 $J(\theta)$

1) 通过度量预测值和目标值的差异生成错误信息。

2) 利用错误信息改变网络权重，获得更精准的预测。

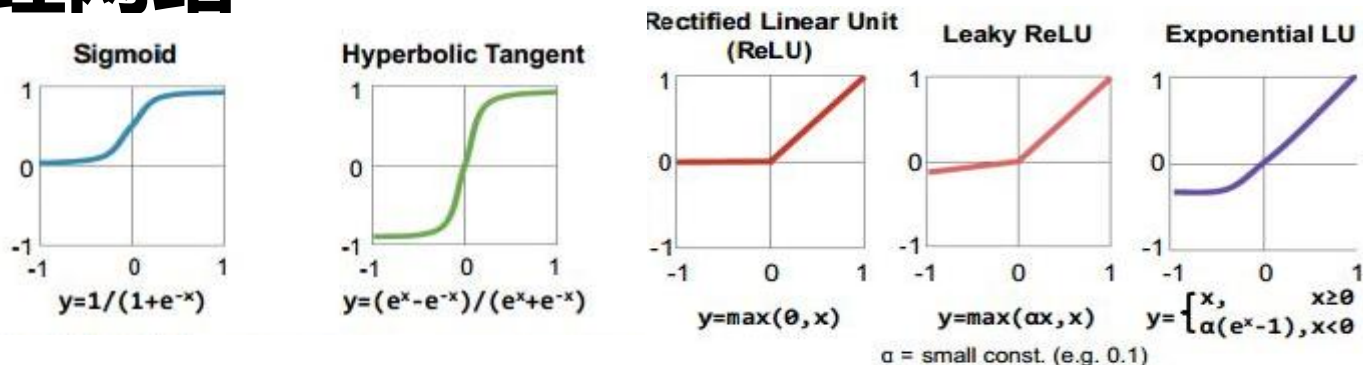
减去梯度的一小部分会使目标函数趋于 (局部) 最小值



<https://medium.com/@ramrajchandradevan/the-evolution-of-gradient-descent-optimization-algorithm-4106a6702d39>

2. 经典神经网络

激活函数



损失函数

平方损失函数，对数损失函数、交叉熵损失函数

$$y = \text{labels} \quad p_{ij} = \text{sigmoid}(\text{logits}_{ij}) = \frac{1}{1 + e^{-\text{logits}_{ij}}}$$

$$\text{loss}_{ij} = -[y_{ij} * \ln p_{ij} + (1 - y_{ij}) \ln(1 - p_{ij})]$$

优化器

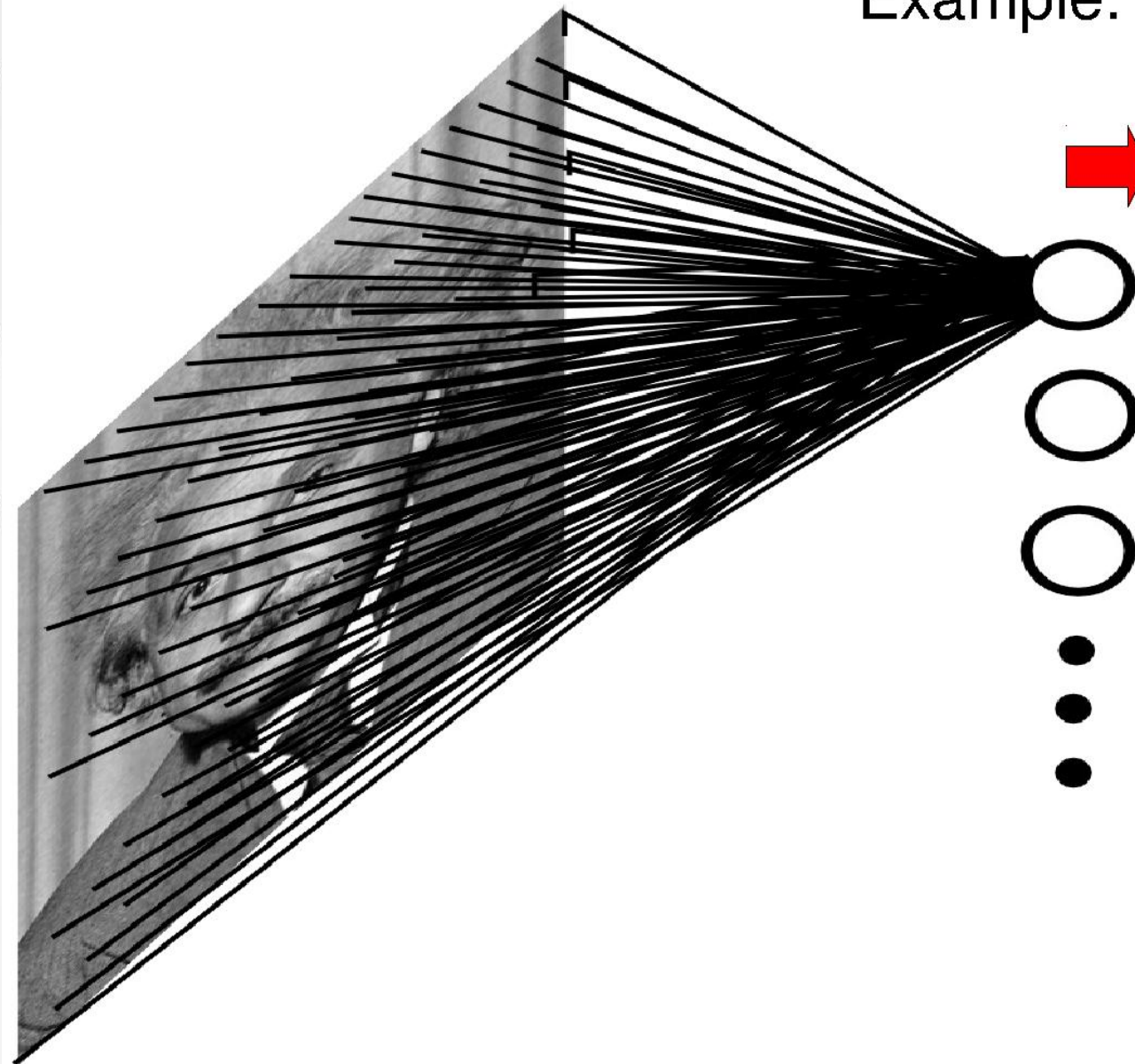
标准梯度下降法(**GD**, Gradient Descent), 随机梯度下降法(**SGD**, Stochastic Gradient Descent) 及批量梯度下降法(**BGD**, Batch Gradient Descent)。

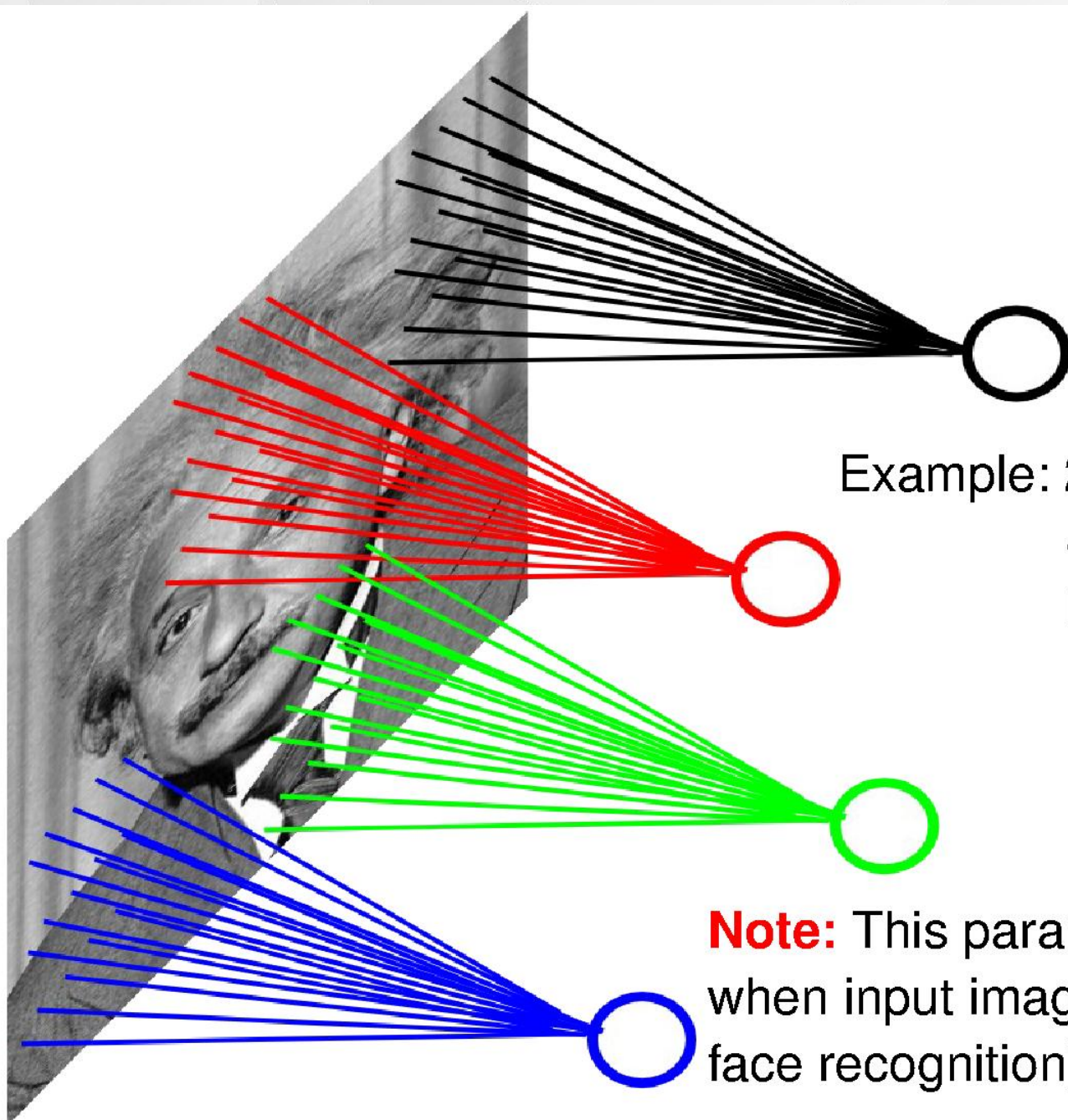
图像输入 神经网络

Example: 200x200 image

40K hidden units

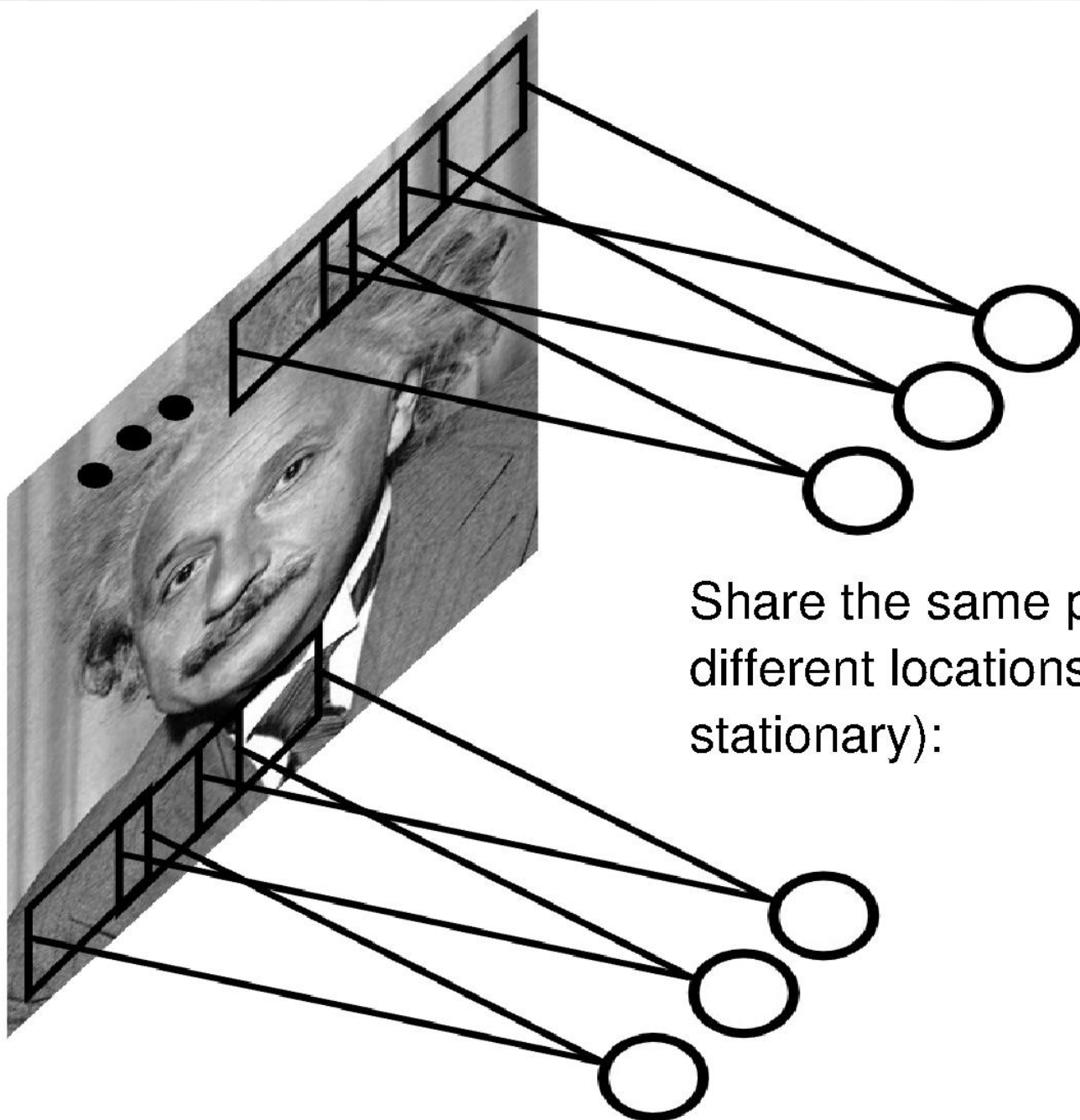
➡ **~2B parameters!!!**





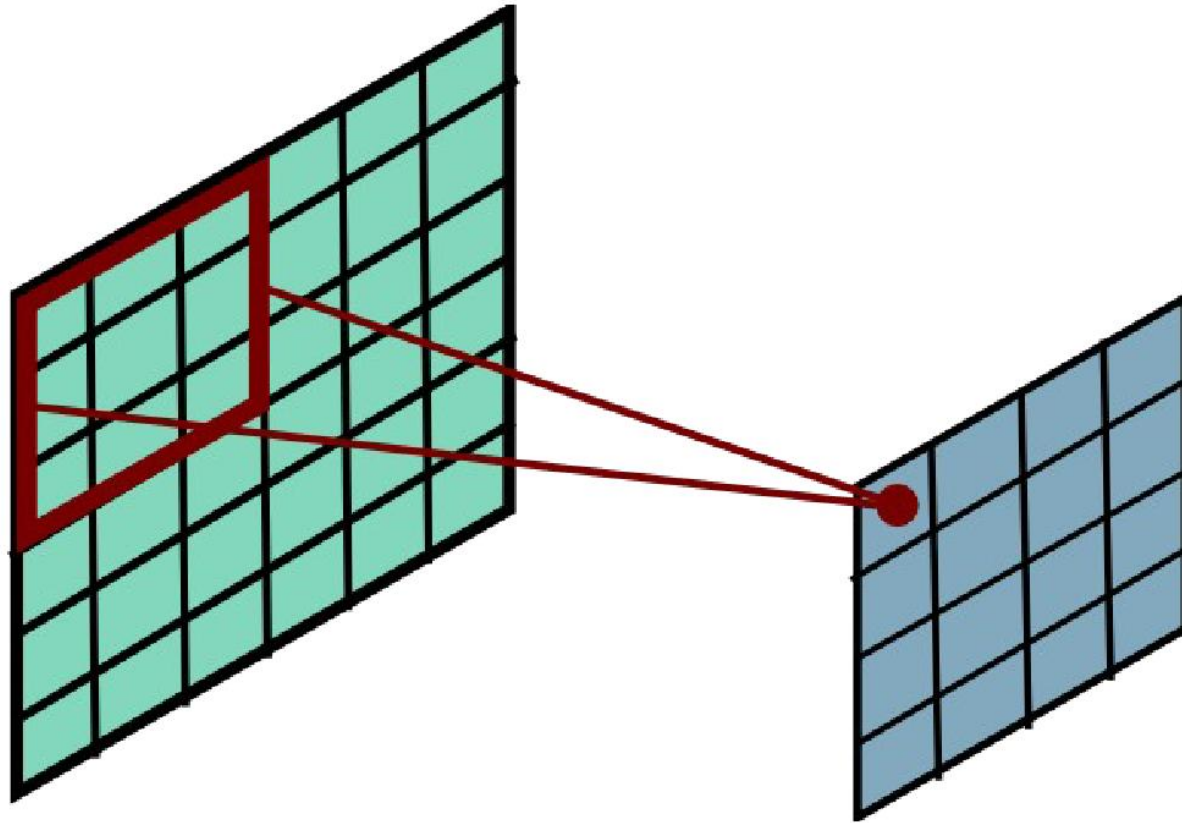
Example: 200x200 image
40K hidden units
Filter size: 10x10
4M parameters

Note: This parameterization is good
when input image is registered (e.g.,
face recognition).

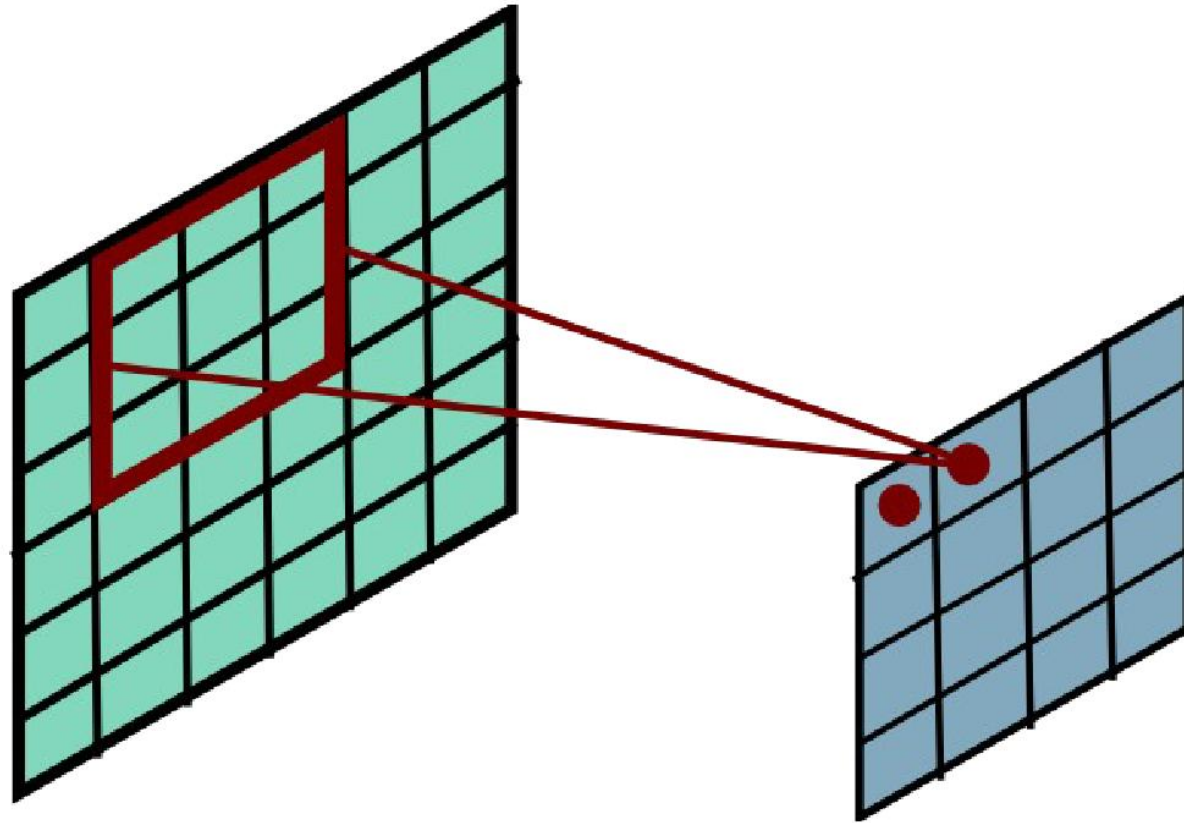


Share the same parameters across different locations (assuming input is stationary):

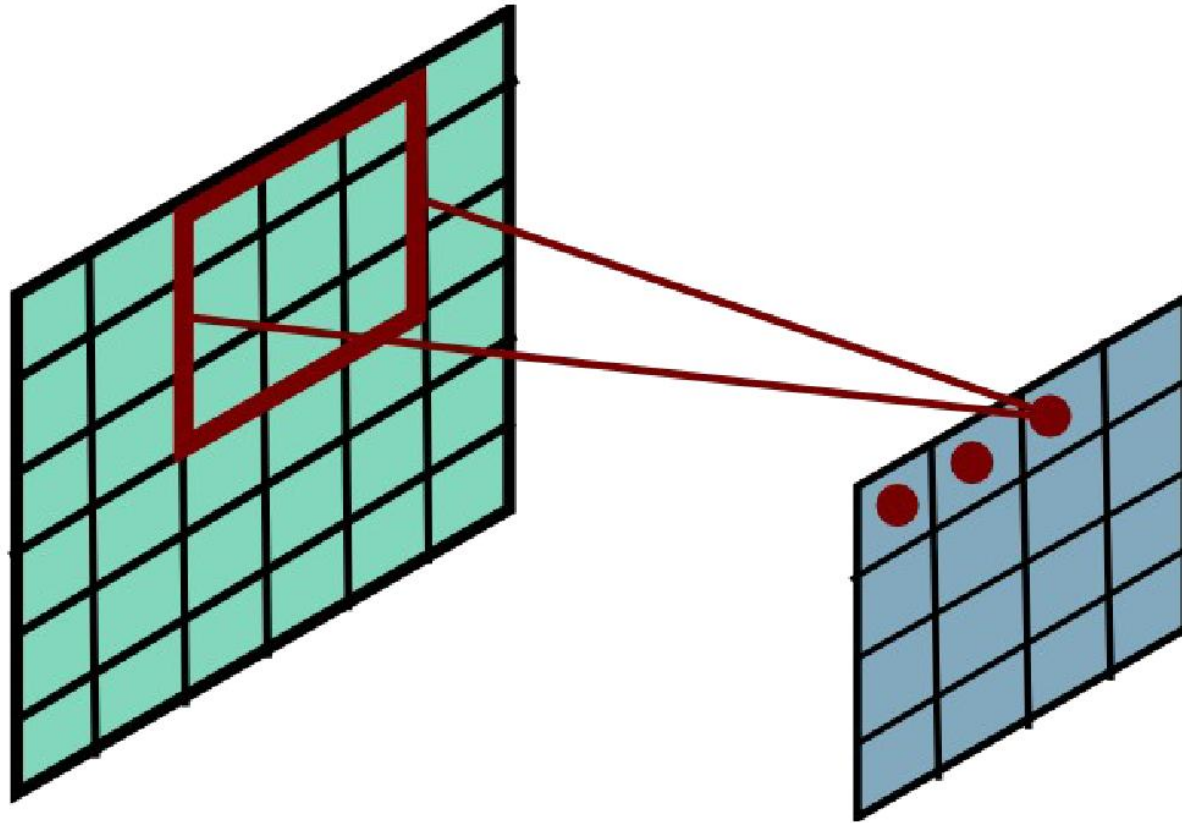
Convolutional Layer



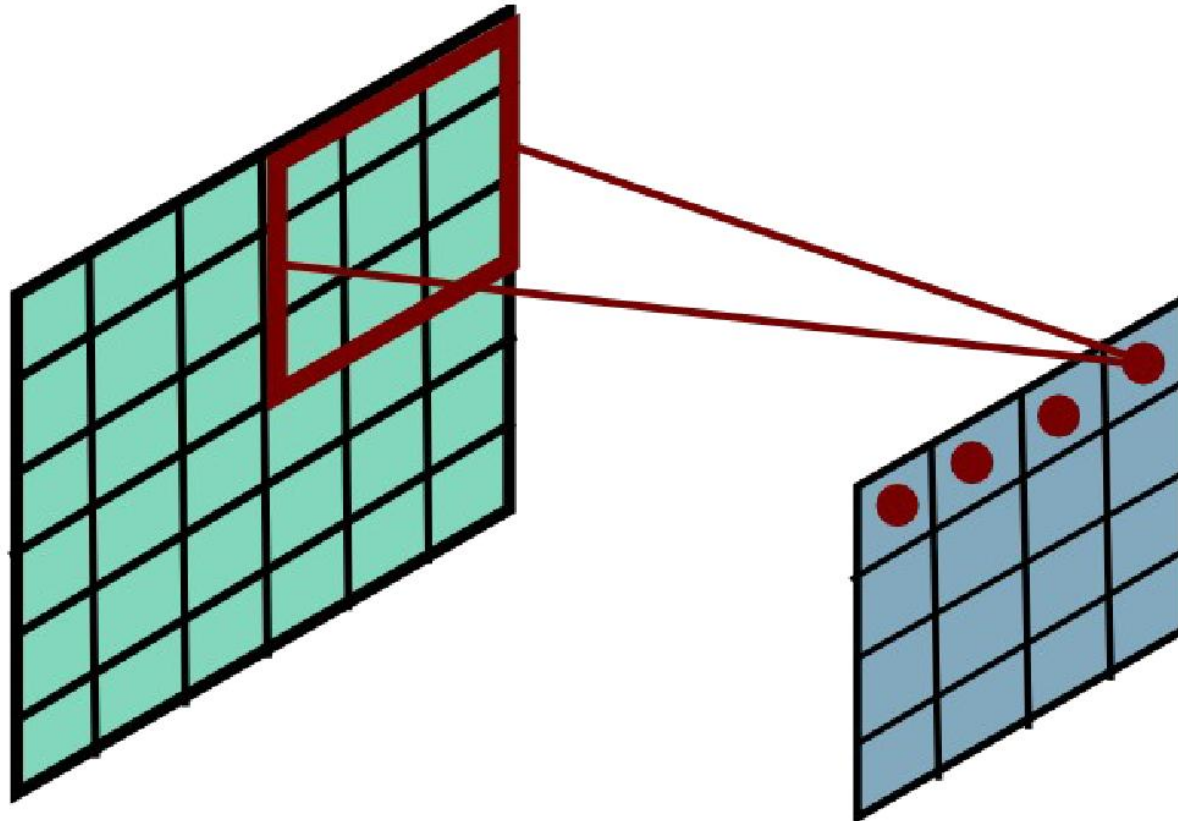
Convolutional Layer



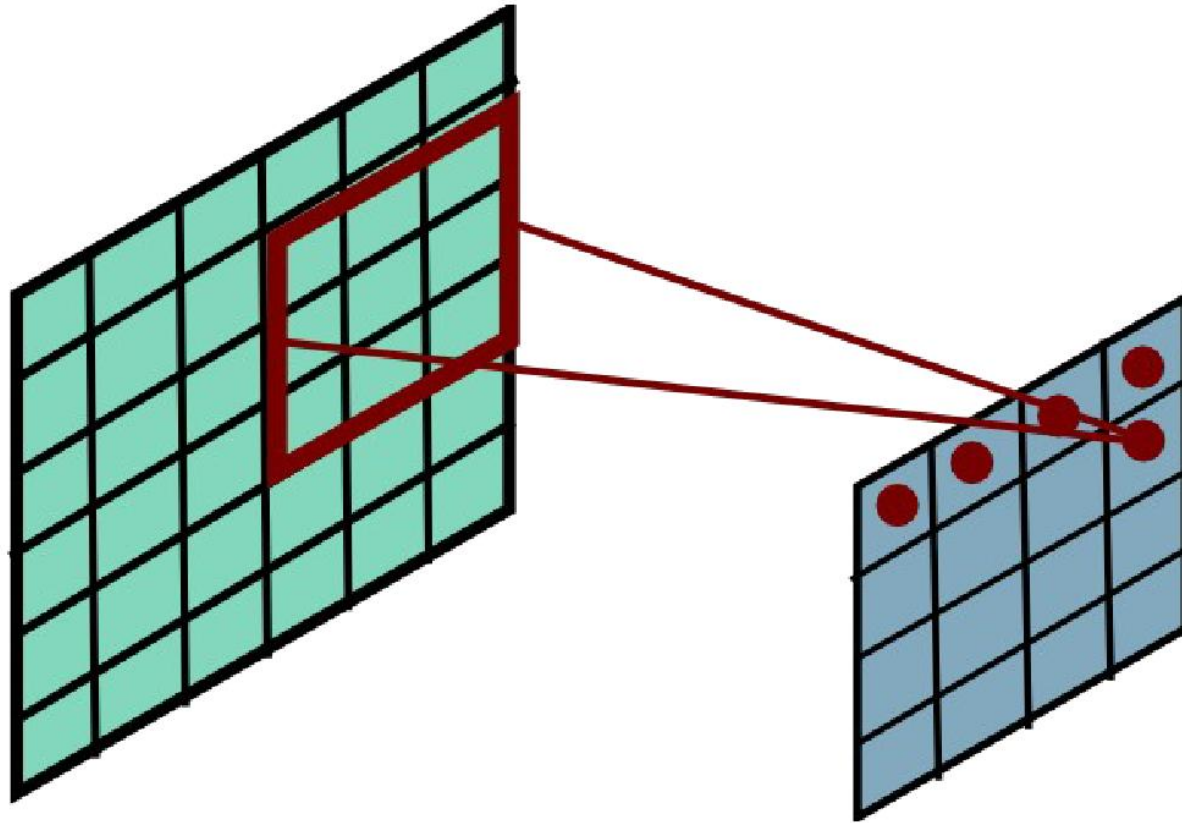
Convolutional Layer



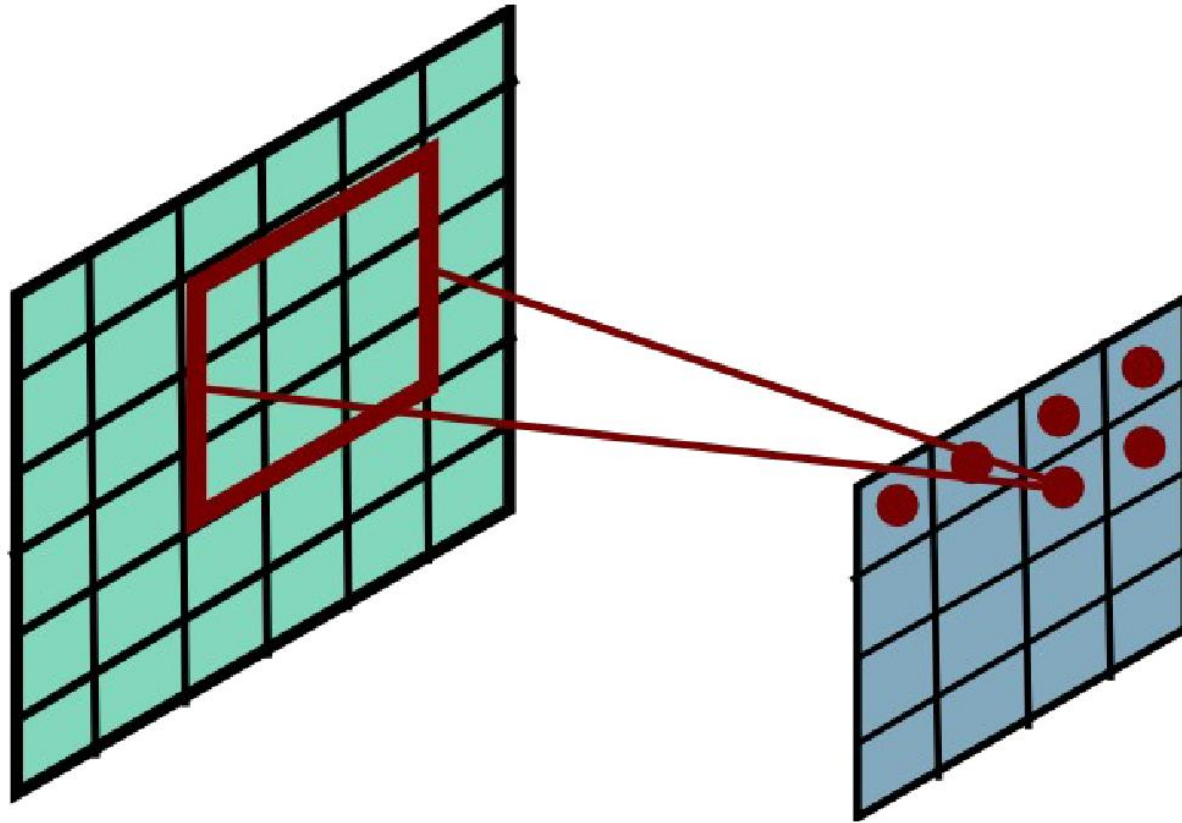
Convolutional Layer



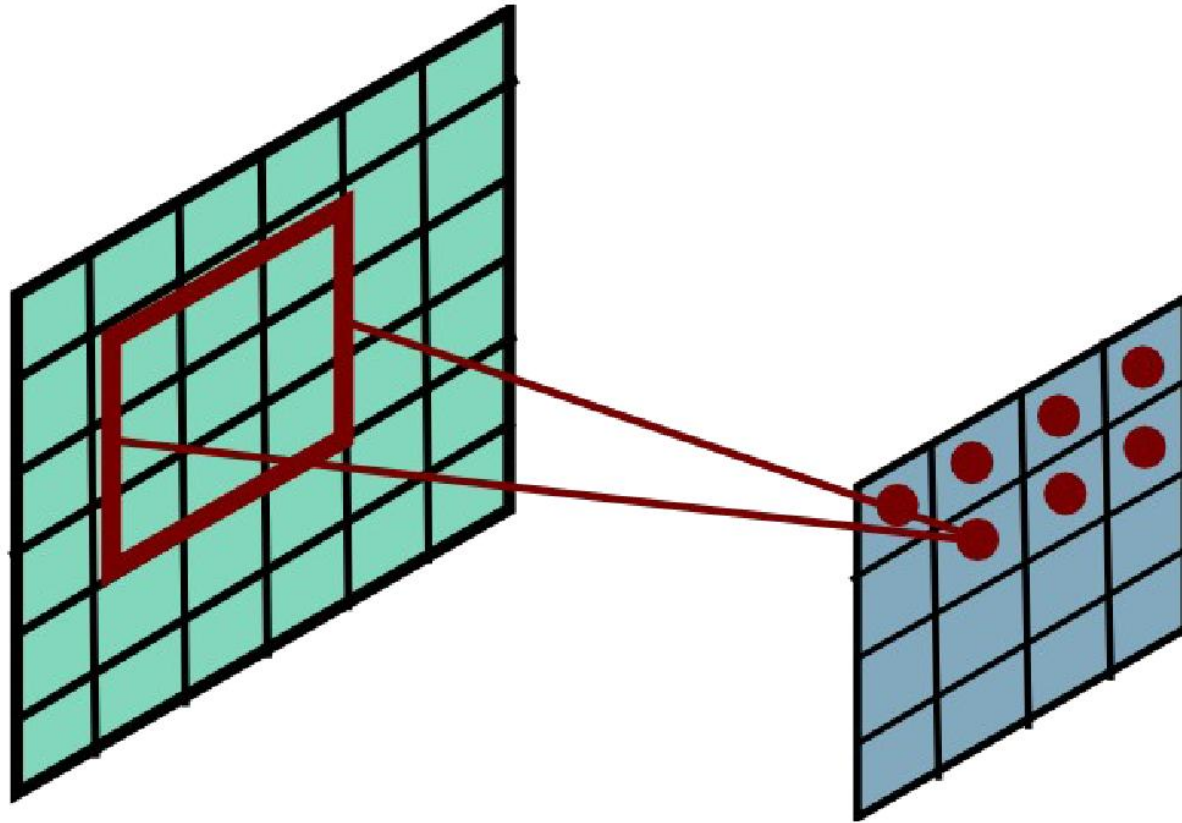
Convolutional Layer



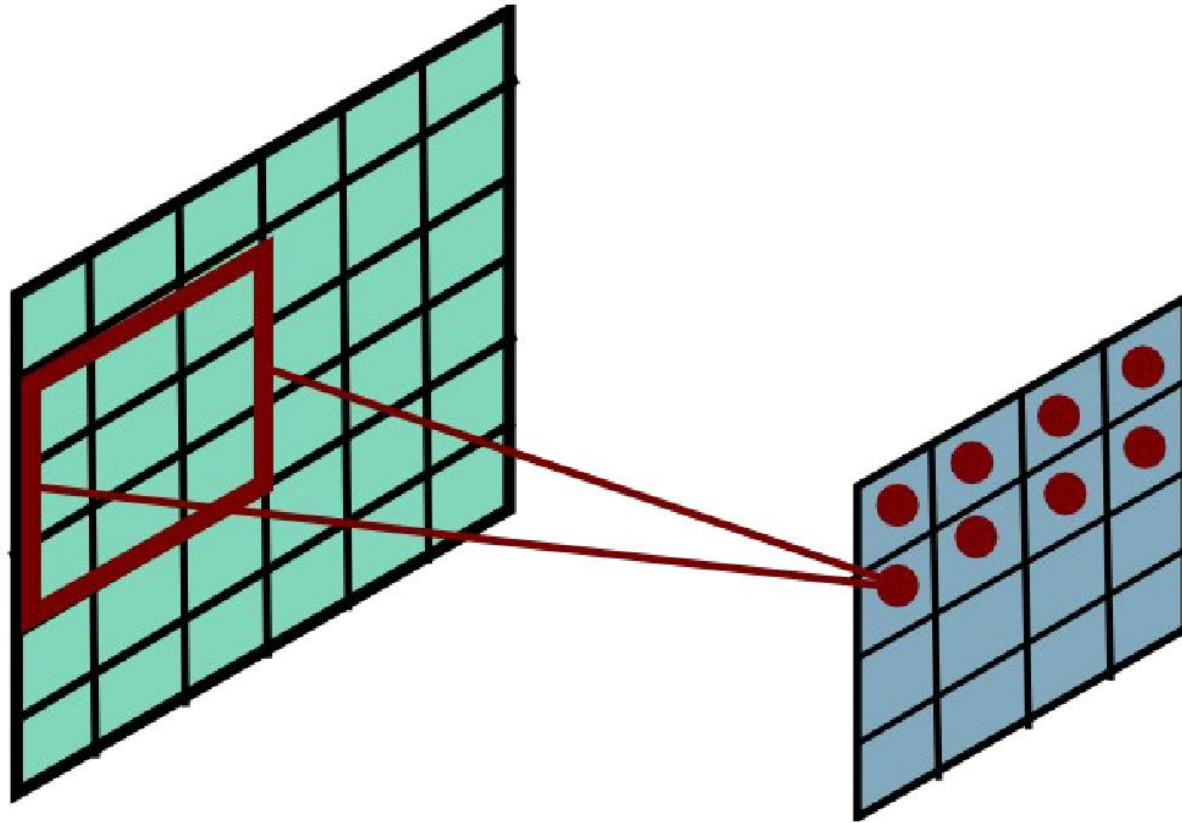
Convolutional Layer



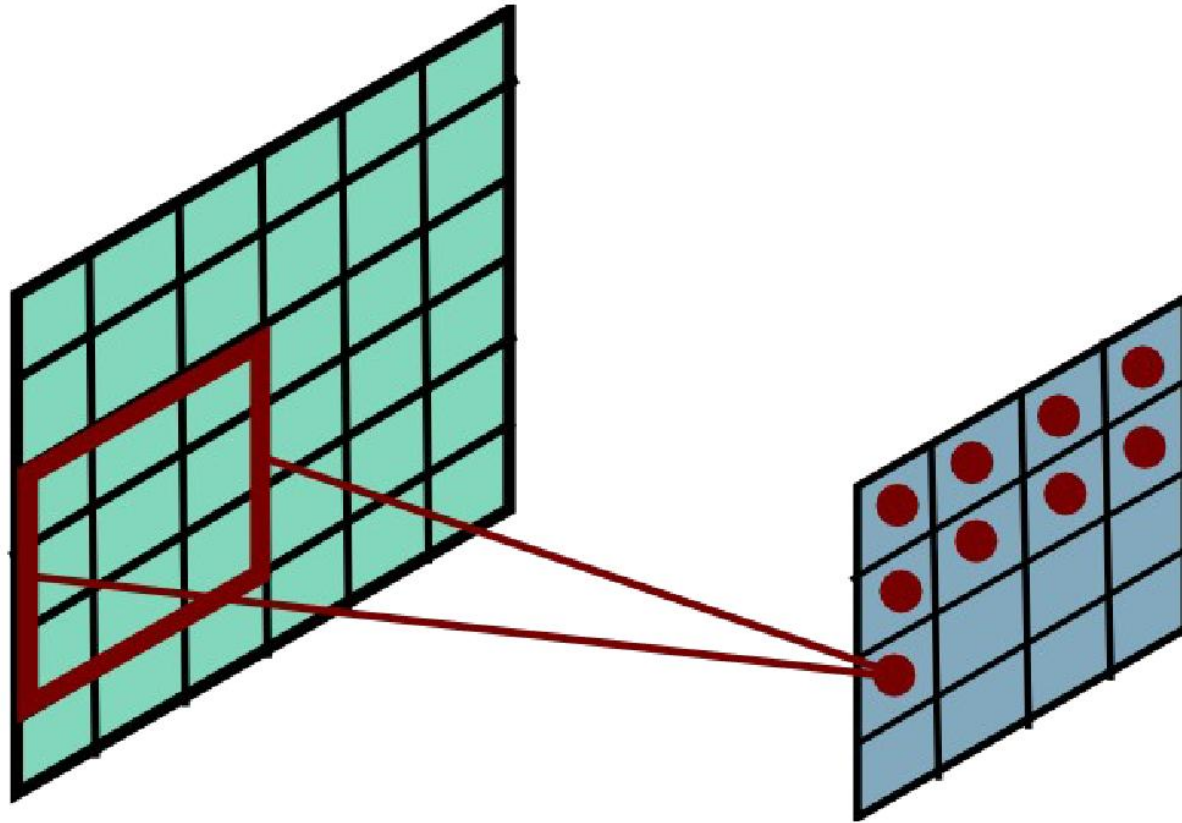
Convolutional Layer



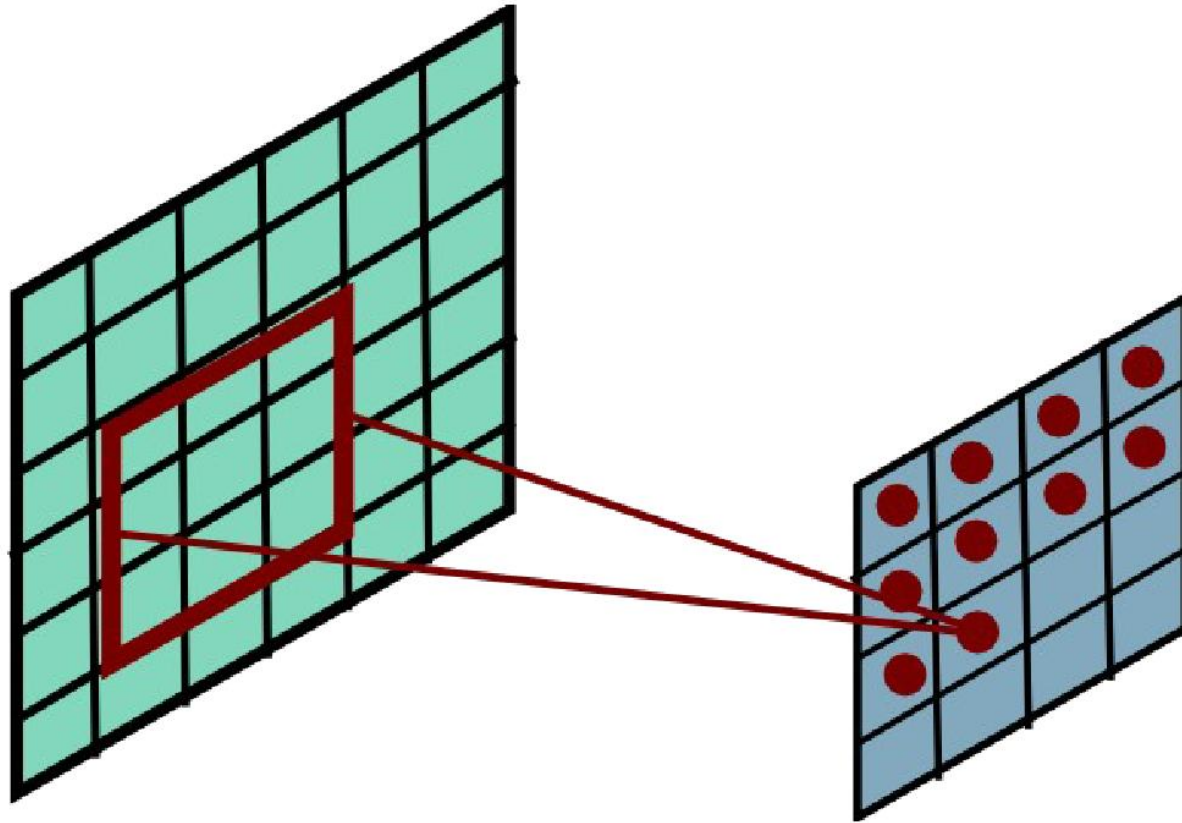
Convolutional Layer



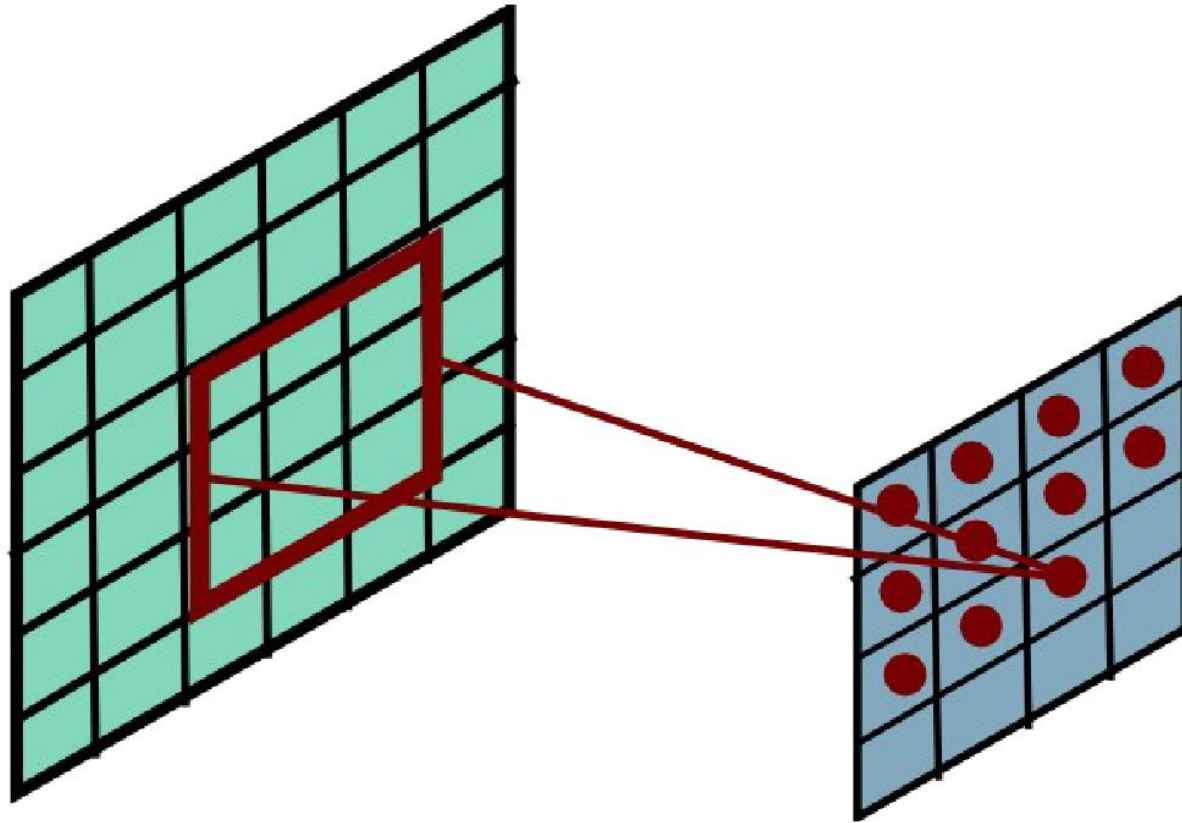
Convolutional Layer



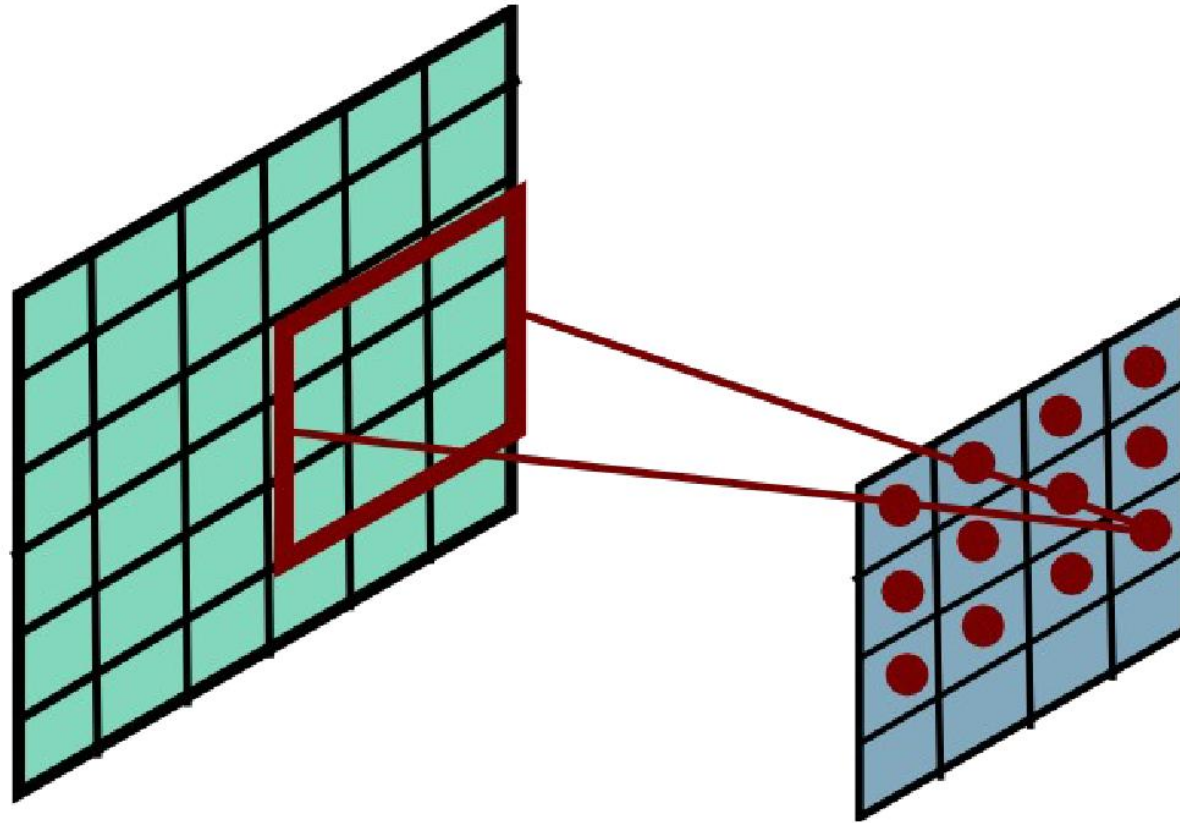
Convolutional Layer



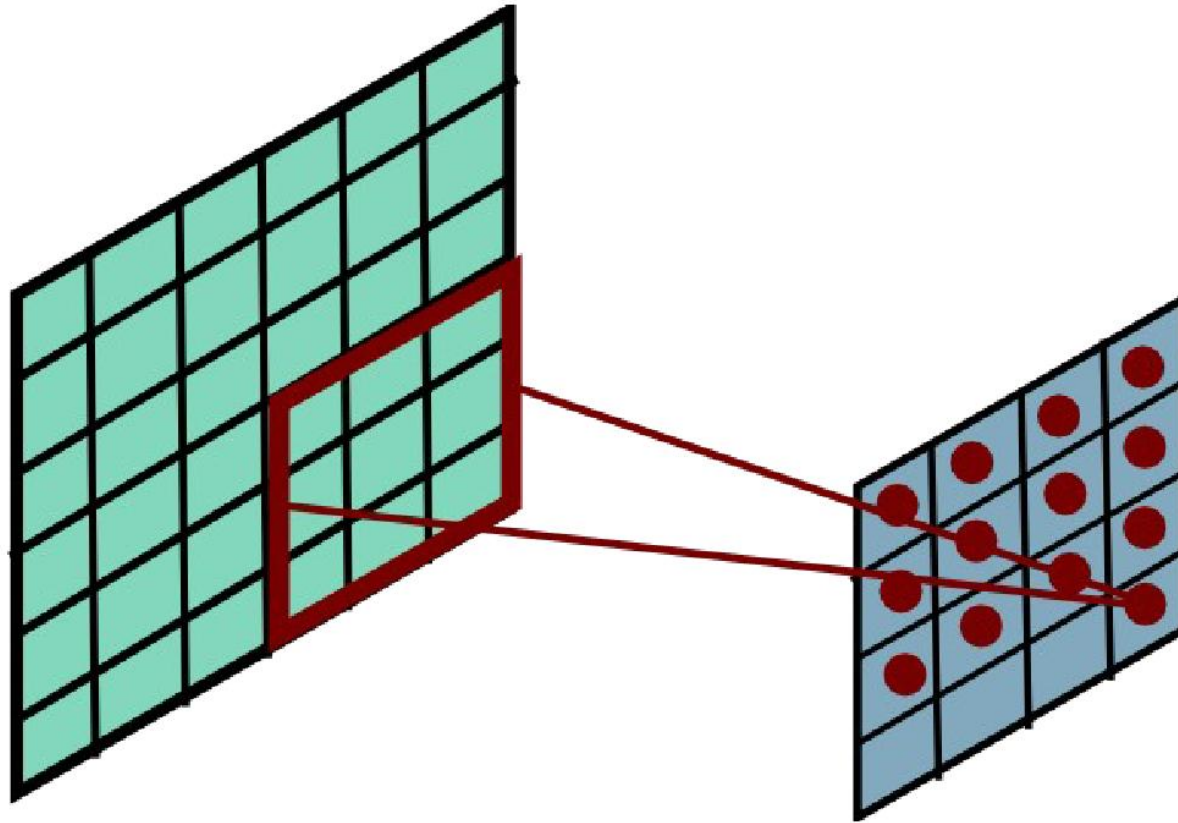
Convolutional Layer



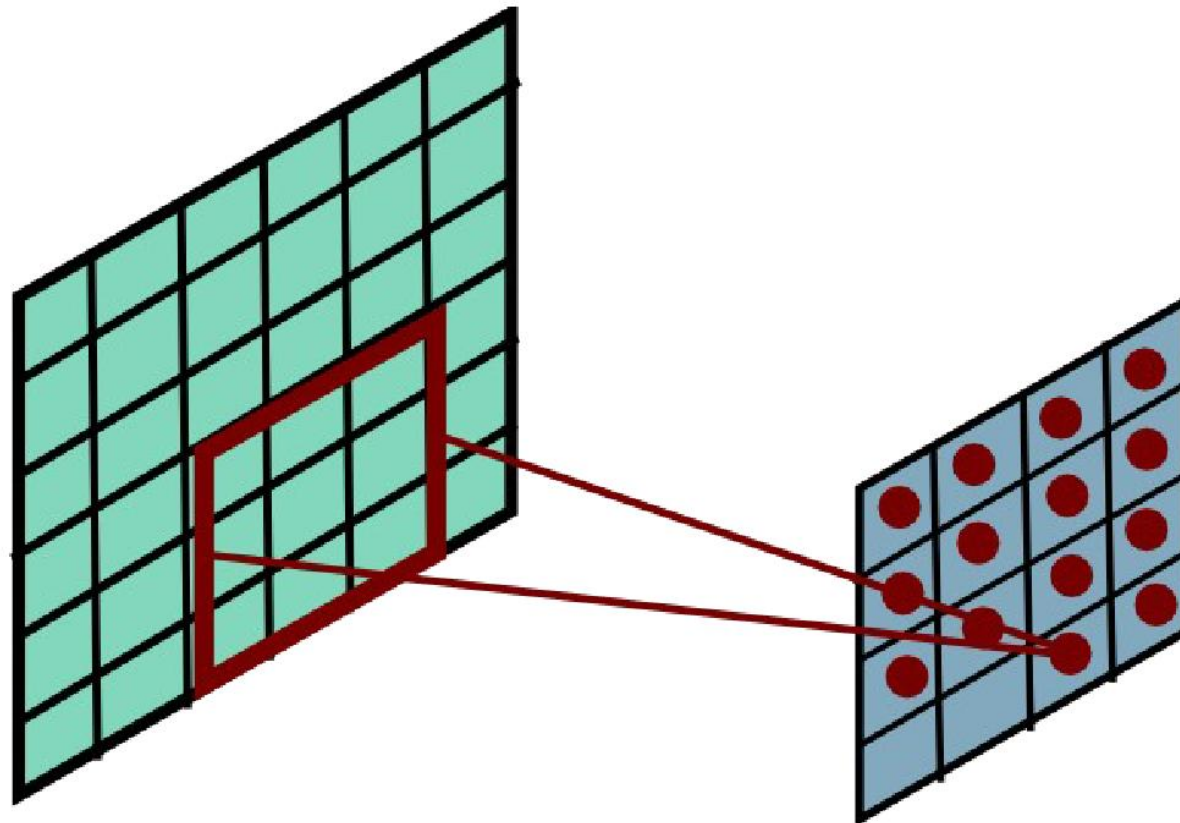
Convolutional Layer



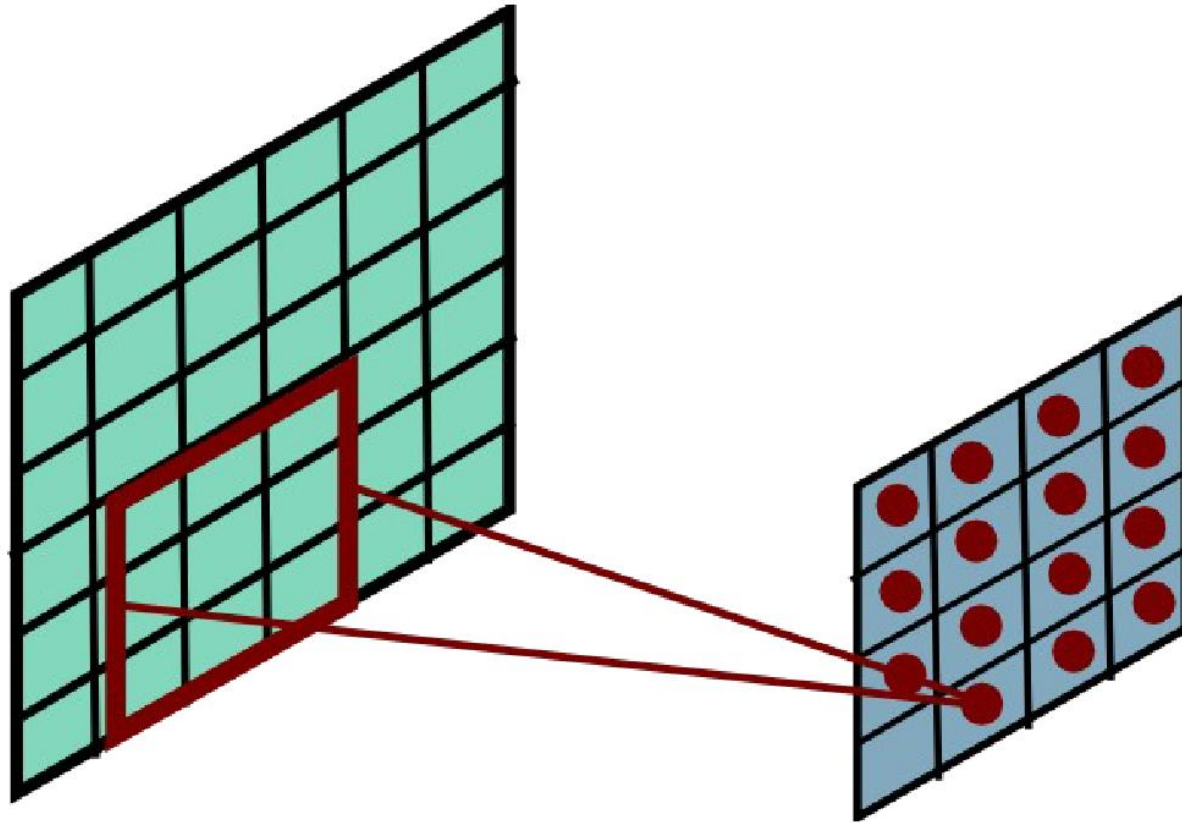
Convolutional Layer



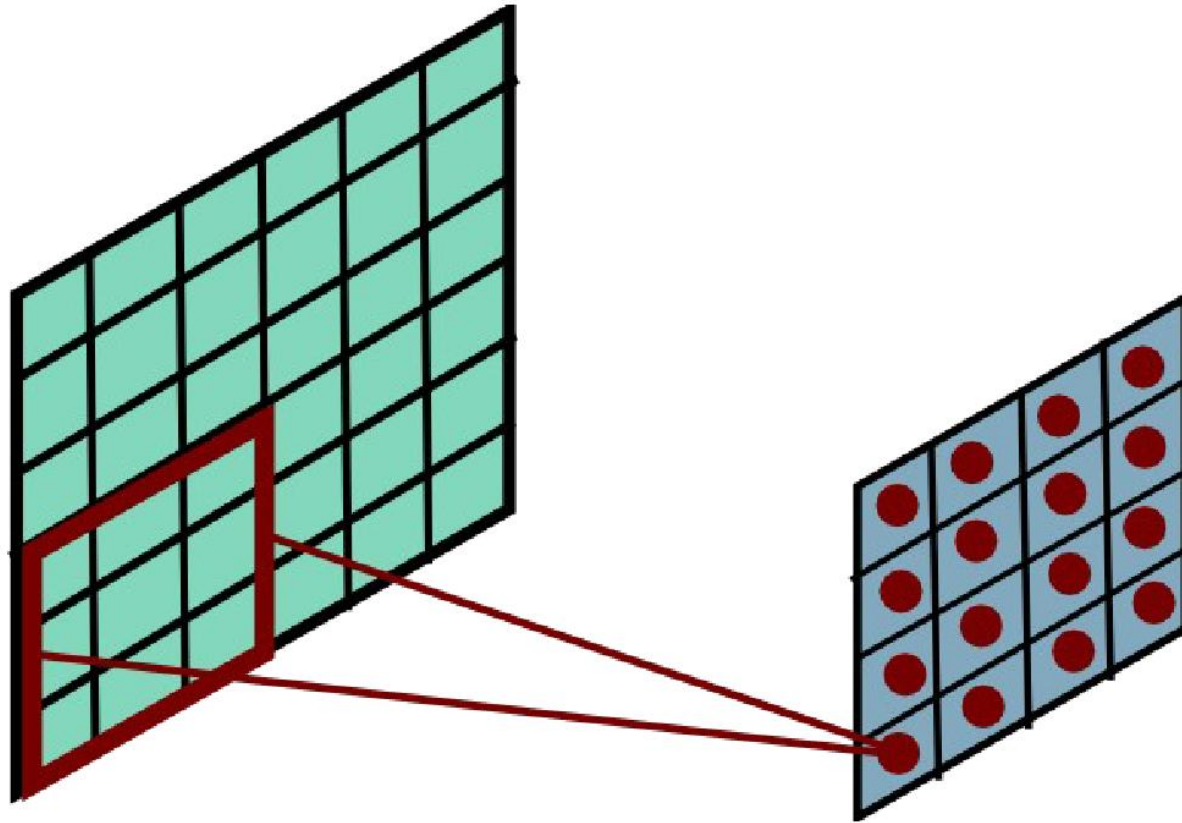
Convolutional Layer



Convolutional Layer



Convolutional Layer



Input Volume (+pad 1) (7x7x3)

W1 (3x3x3)

Output Volume (3x3x2)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	0	0	1	1	0	0
0	0	0	2	2	0	0
0	0	1	1	0	2	0
0	2	2	2	0	0	0
0	0	0	1	2	0	0
0	0	0	0	0	0	0

 $w0[:, :, 0]$

-1	-1	0
1	1	1
-1	-1	-1

 $w1[:, :, 0]$

-1	1	0
-1	1	-1
-1	-1	-1

 $o[:, :, 0]$

-1	-3	0
-4	-8	-2
-7	-7	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	1	0	0	1	0	0
0	0	0	2	0	0	0
0	1	0	0	0	0	0
0	1	1	2	1	0	0
0	1	2	1	2	2	0
0	0	0	0	0	0	0

 $w0[:, :, 2]$

1	-1	1
-1	-1	-1
-1	0	0

 $w1[:, :, 2]$

1	-1	0
0	0	-1
0	-1	-1

 $o[:, :, 1]$

-2	-5	-5
-6	-5	-3
-1	-8	-6

Bias $b0$ (1x1x1) $b0[:, :, 0]$

1

Bias $b1$ (1x1x1) $b1[:, :, 0]$

0

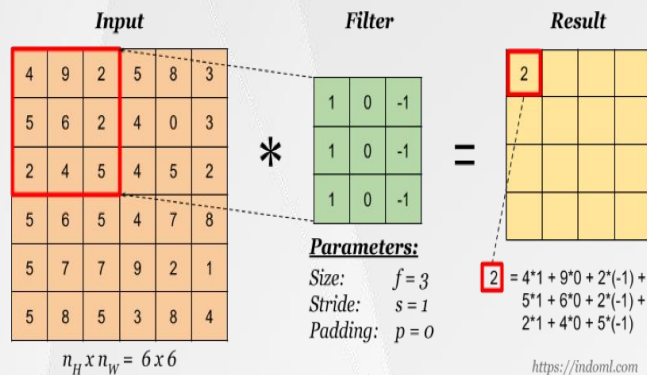
 $x[:, :, 2]$

0	0	0	0	0	0	0
0	1	1	2	0	0	0
0	0	0	1	1	1	0
0	2	1	2	1	0	0
0	1	0	2	1	2	0
0	1	0	0	2	1	0
0	0	0	0	0	0	0

toggle movement

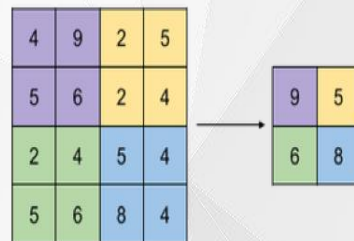
CNN各种

layer

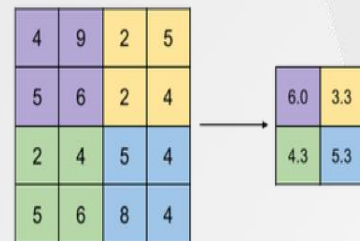


卷积层

Max Pooling

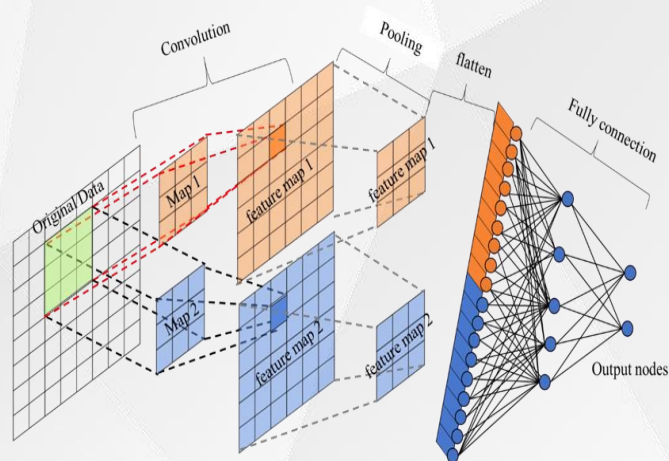


Avg Pooling



<https://indoml.com>

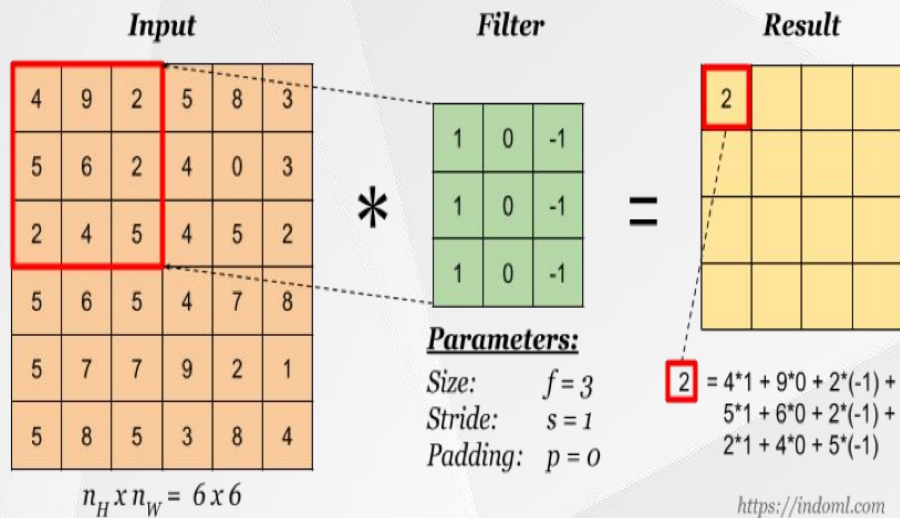
池化层



全连接层

- 一般的CNN结构为 (卷积-池化)*n - 全连接
- 卷积层：获取图像的局部特征
- 池化层：减少参数量，防止过拟合
- 全连接层：将局部特征组装成完整的图

卷积层

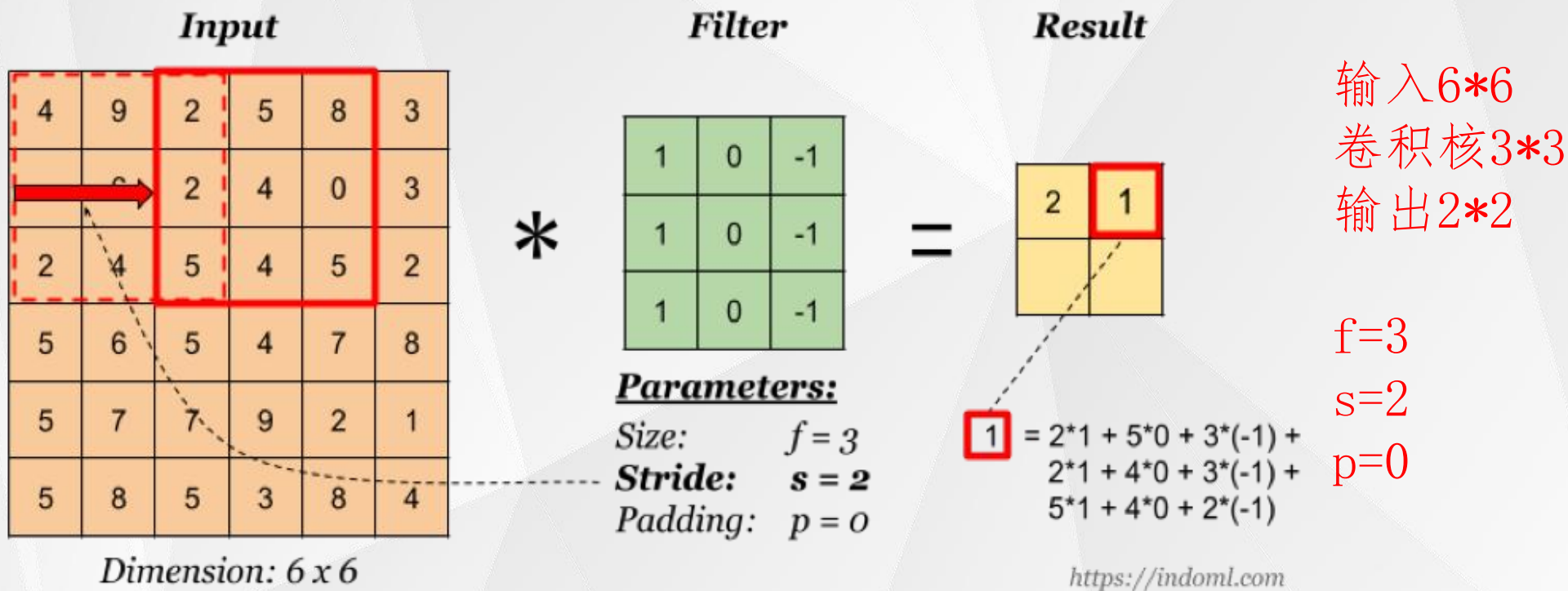


输入 6×6
卷积核 3×3
输出 4×4

$f=3$
 $s=1$
 $p=0$

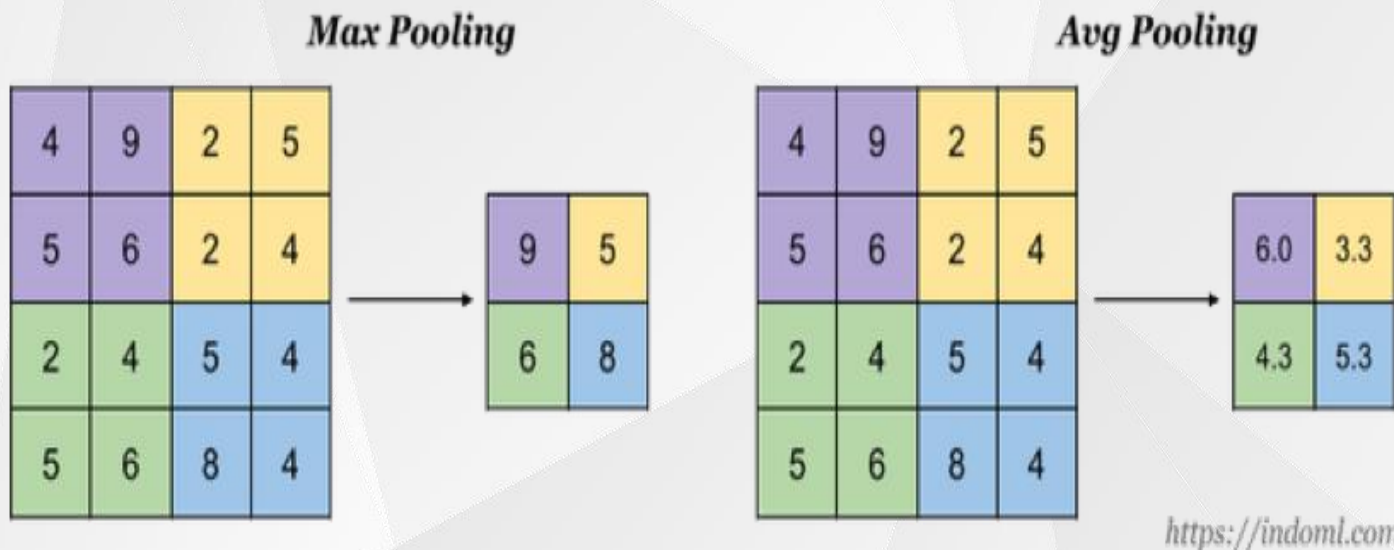
- 通过输入图片与卷积核卷积得到输出
- 卷积操作为对应位置元素相乘最后相加
- 通过卷积核不断滑动来实现参数共享
- 输出的大小与卷积核的Size, Padding, Stride相关

卷积层Stride



- **Stride**为卷积核的步长，代表一次滑动多少单元
- 通过卷积核不断滑动来实现参数共享
- 滑动窗覆盖处原图与卷积核进行卷积输出结果

池化层



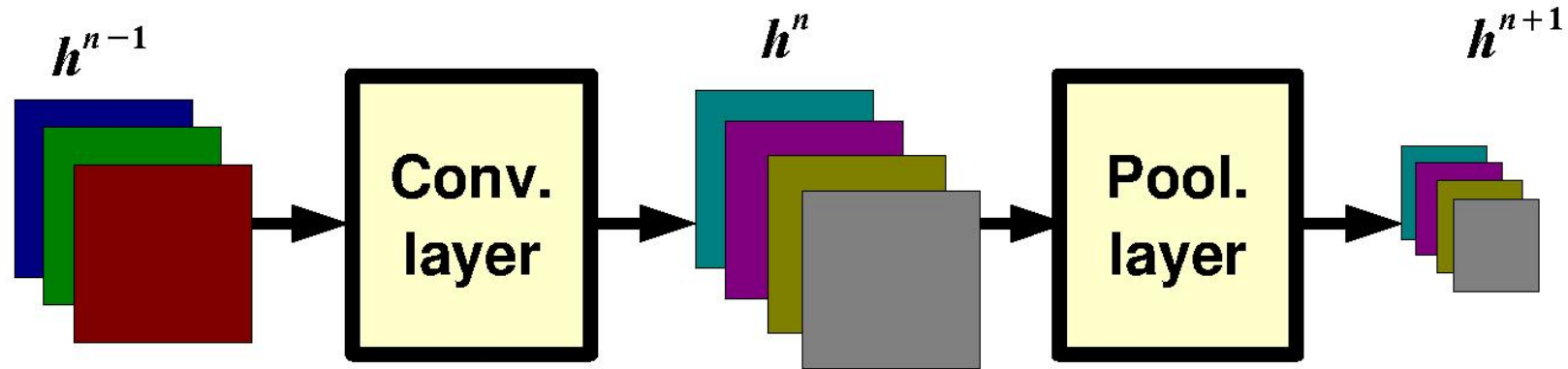
- 池化操作为下采样，降低了后续网络层的输入维度，从而减少计算量
- 池化操作Stride为2，一般多采用2*2的采样窗口
- Average Pooling 返回窗口内元素的平均值
- Max Pooling 返回窗口内元素的最大值

池化层

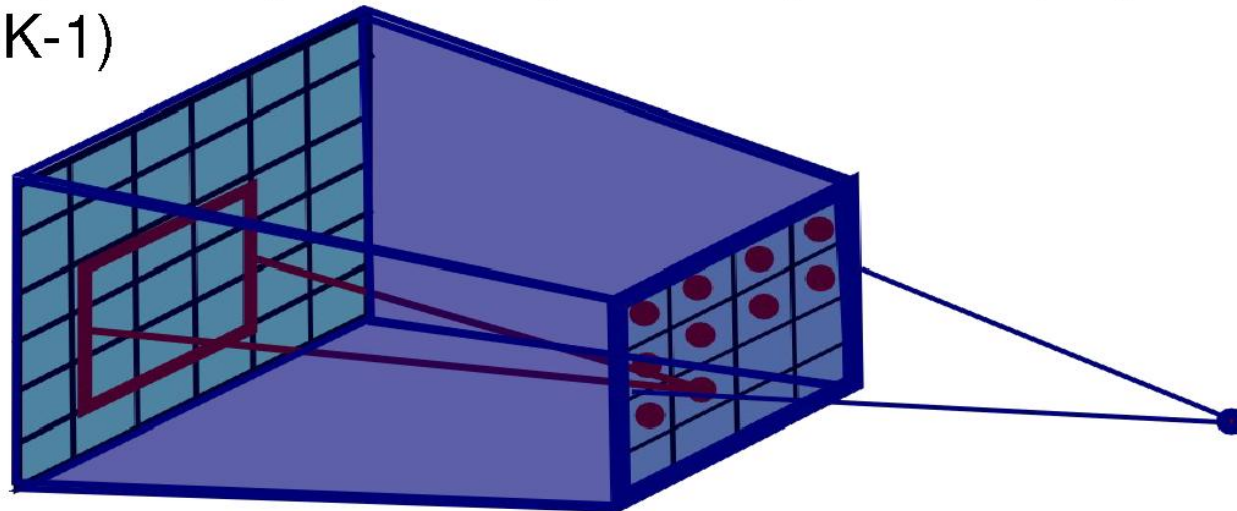


- 池化操作为下采样，降低了后续网络层的输入维度，从而减少计算量
- 池化操作Stride为2，一般多采用2*2的采样窗口
- Average Pooling 返回窗口内元素的平均值
- Max Pooling 返回窗口内元素的最大值

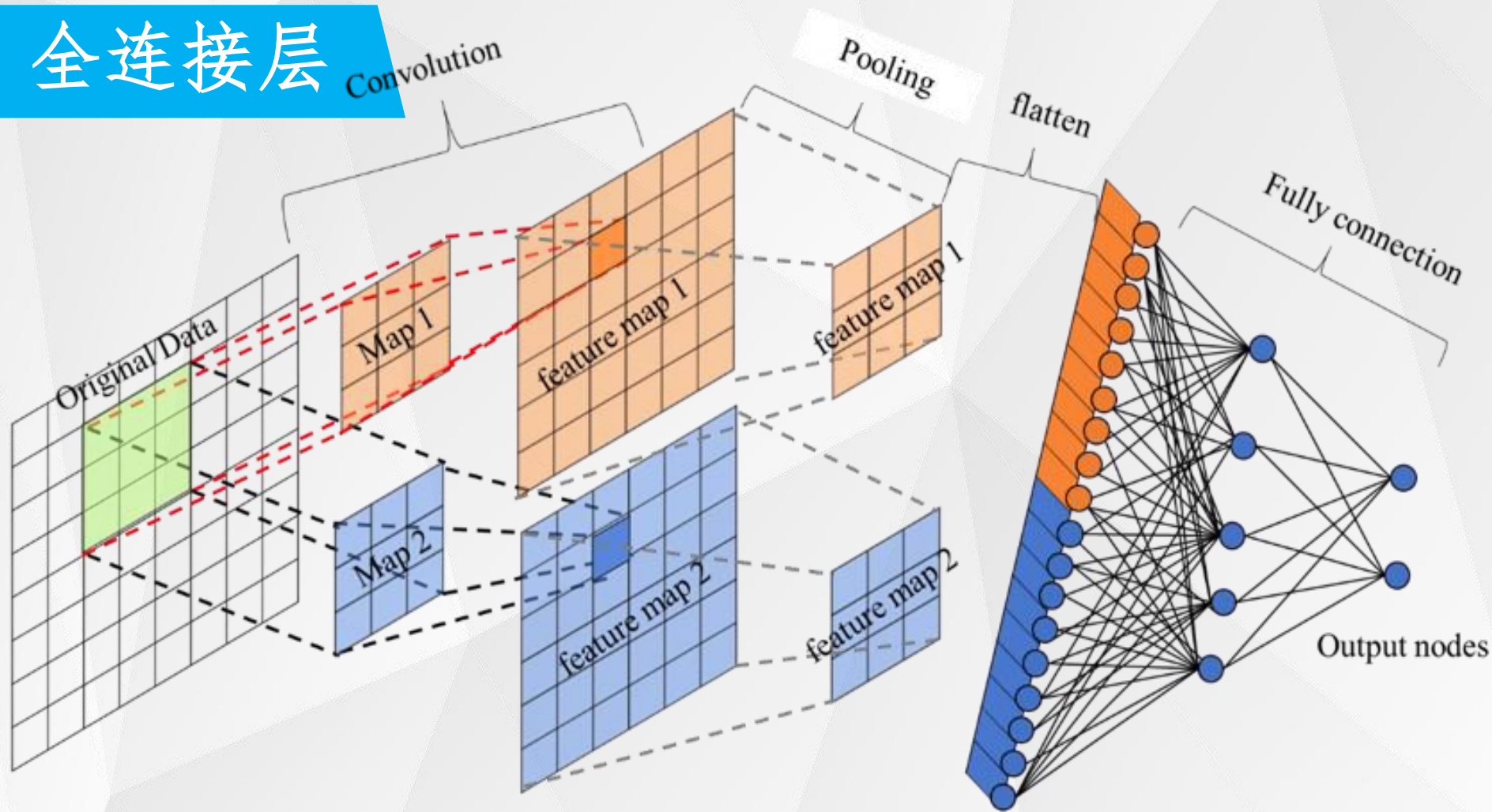
Pooling Layer: Receptive Field Size



If convolutional filters have size $K \times K$ and stride 1, and pooling layer has pools of size $P \times P$, then each unit in the pooling layer depends upon a patch (at the input of the preceding conv. layer) of size: $(P+K-1) \times (P+K-1)$



全连接层



- 全连接层将获取到的局部特征重组
- 原理其实是用很多个与InputSize相同的卷积核进行卷积

4. 深度学习平台

常用数据集

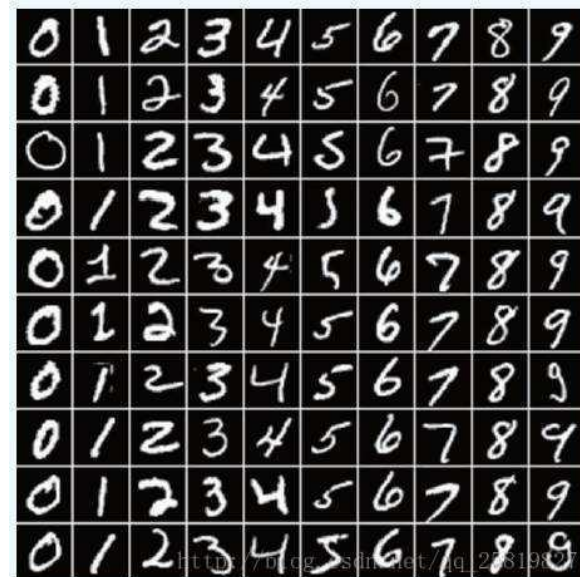
(一) mnist手写字体数据集(分类):

mnist数据是由Google实验室的Corinna Cortes和纽约大学柯朗研究所的LeCun建立的一个手写字体数据集，其中训练集包含60000训练的手写数字图片，测试集包含10000张图片，一个训练集的标签集，一个测试集的标签集。

官方地址：

<http://yann.lecun.com/exdb/mnist/>

数据集中图片为单通道，大小为28*28像素
基本上正确率都能达到98%+，适合初学者上手。



4. 深度学习平台

常用数据集

(二) ImageNet数据集(分类):

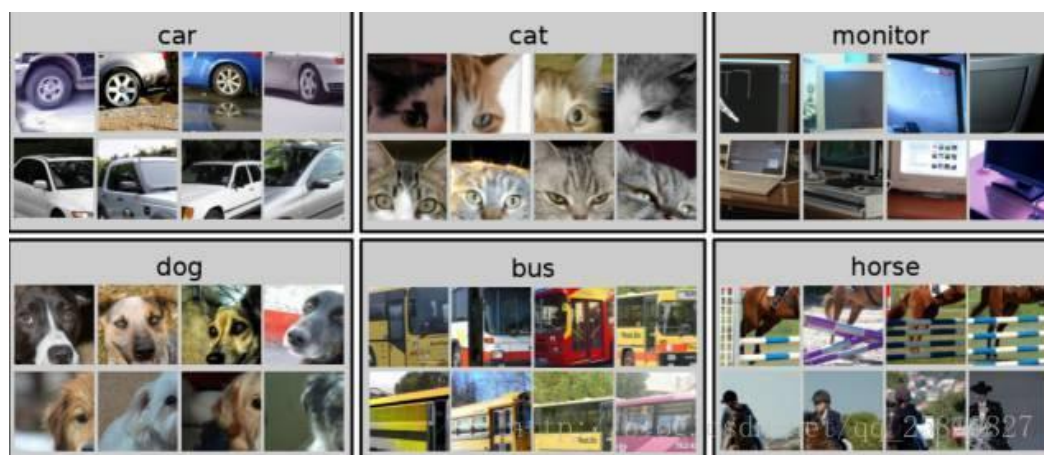
ImageNet数据是由Stanford University 李飞飞实验室主导发布，数据集中有1400万幅图像，涵盖了2万多个类别；其中有超过百万的图片有明确的类别标注和图像中物体位置的标注。



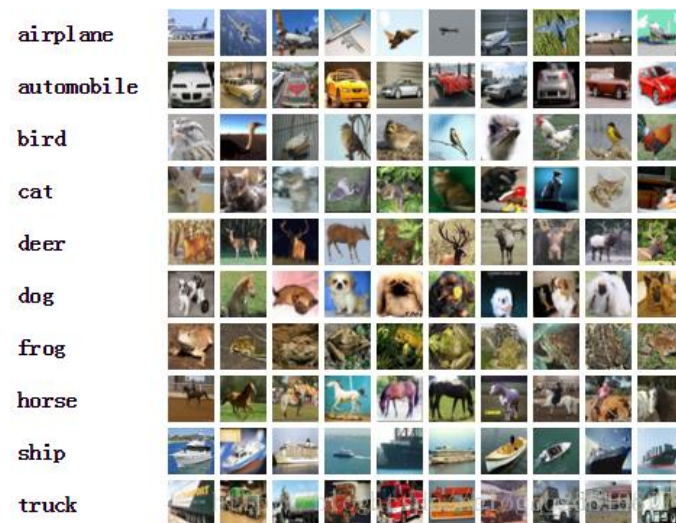
4. 深度学习平台

常用数据集

(三) PASCAL VOC 分类识别与检测



(四) cifar-分类



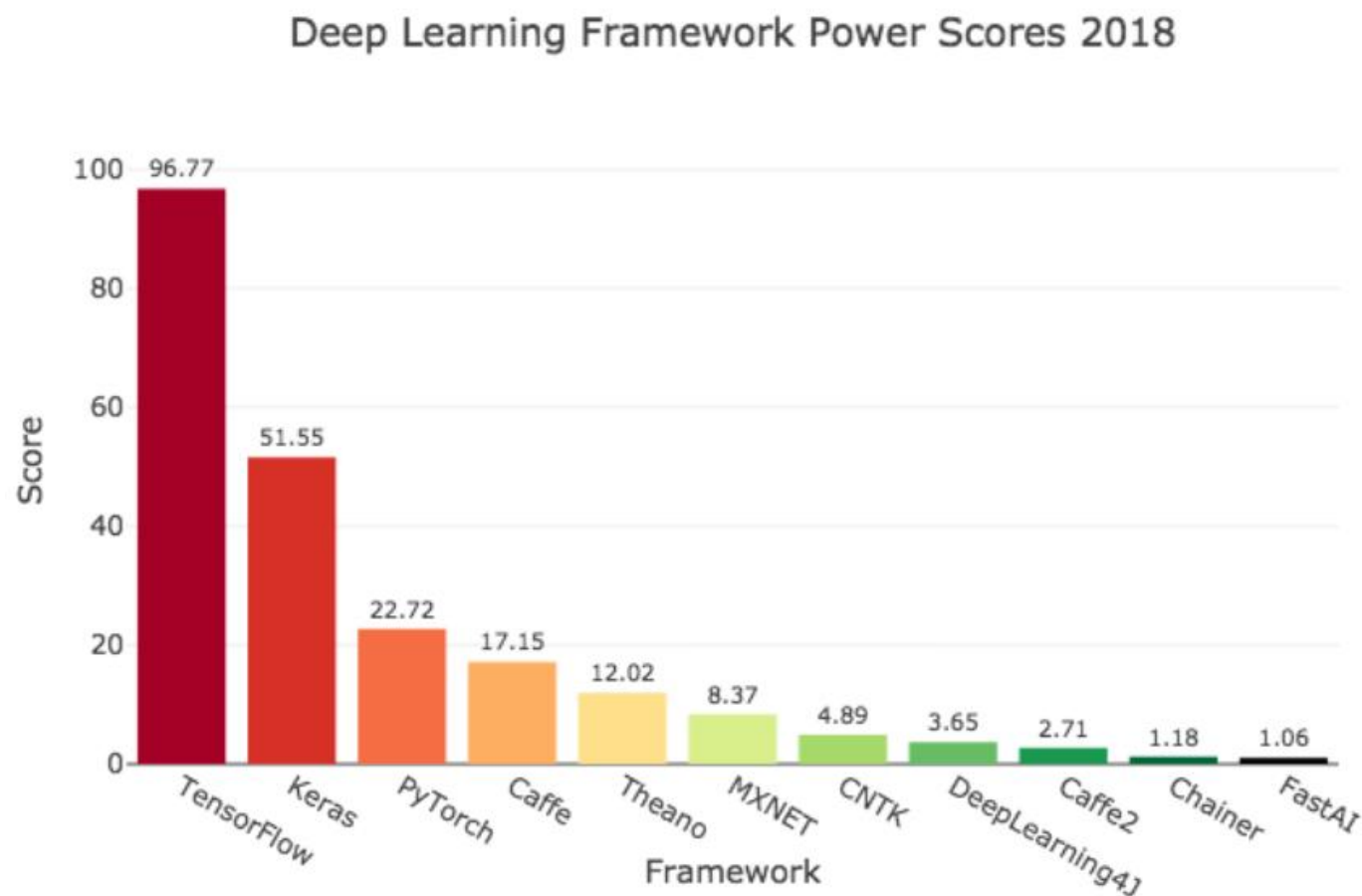
(五) COCO-图像分割



4. 深度学习平台

常用平台

 A chart from 2018 showing some popular DL frameworks. Source: Hale 2018.*



4. 深度学习平台

Keras



- Tensorflow 和 Theano接口封装.
- 作者为Google公司职员Francois Chollet, Keras 将变成Tensorflow API.
- 官方文档: <http://keras.io>.
- 实例:
<https://github.com/fchollet/keras/tree/master/examples>
- Tensorflow简单教程:
<https://docs.google.com/presentation/d/1zkmVGobdPfQgsjlw6gUqJsJB8wvv9uBdT7ZHdaCjZ7Q/edit#slide=id.p>

4. 深度学习平台

平台安装

安装深度学习工具包：

tensorflow (深度学习库)

keras (Python封装深度学习框架)

CPU 版本

```
>>> pip install --upgrade tensorflow
```

Keras 安装

```
>>> pip install keras -U --pre
```

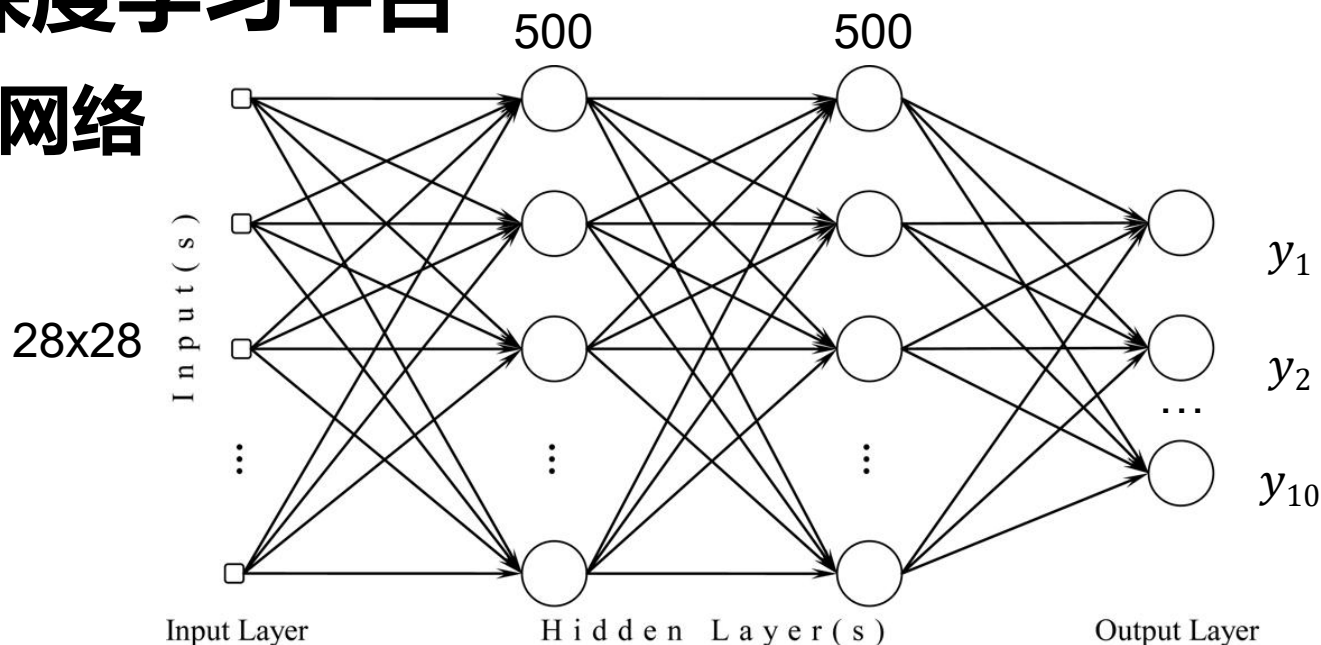
之后可以验证keras是否安装成功,在命令行中输入Python命令

进入Python变成命令行环境: >>> import keras

没有报错, 那么Keras就已经**成功安装**

4. 深度学习平台

搭建网络



```
model = sequential() # layers are sequentially added
model.add( Dense(input_dim=28*28, output_dim=500))
model.add(Activation( 'sigmoid' )) #: softplus, softsign,relu,tanh, hard_sigmoid
model.add(Dense( output_dim = 500))
model.add (Activation( 'sigmoid' ))
Model.add(Dense(output_dim=10))
Model.add(Activation('softmax'))
model.compile(loss= 'categorical_crossentropy' , optimizer= 'adam' , metrics=[ 'accuracy' ])
model.fit(x_train, y_train, batch_size=100, nb_epoch=20)
```

4. 深度学习平台

搭建模式

序贯模型 (Sequential) : 多个网络层的堆叠 (add)

For a single-input model with 2 classes (binary classification):

```
model = Sequential()  
model.add(Dense(32, activation='relu', input_dim=100))  
model.add(Dense(1, activation='sigmoid'))  
model.compile(optimizer='rmsprop',
```

函数式模型 (Model) 用户定义多输出模型

```
inputs = Input(shape=(784,))  
x = Dense(64, activation='relu')(inputs)  
x = Dense(64, activation='relu')(x)  
predictions = Dense(10, activation='softmax')(x)  
model = Model(inputs=inputs, outputs=predictions)  
model.compile(optimizer='rmsprop',  
loss='categorical_crossentropy', metrics=['accuracy'])  
model.fit(data, labels) # starts training
```

